AURIX

Microcontroller Infineon Software Architecture

User's Manual

MCU driver

Released v5.2

2017-01

Microcontrollers

Edition 2017-01

Published by
Infineon Technologies AG
81726 Munich, Germany

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (**www.infineon.com**).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Date	Version	Change Description
2017-02-02	5.2	Updated the date format in the version history which was not aligned correctly based on JIRA 0000054991-85. Updated section 6.2.4 based on review comments from GTM review REV_011542.
2016-12-07	5.1	Updated to fix the review comments as per REV_011290. The sections which are updated as per REV_011290 : 1. “Change description” in version history table. 2. Rephrased contents in the Section, titled as “Container:McuPublishedInformation” Note: “McuPublishedInformation” section is applicable only to AS403.
2016-11-30	5.0	Updated the sub-section, titled as “Container:McuPublishedInformation” (under Section 4), based on JIRA 51047-309 Note: “McuPublishedInformation” section is applicable only to AS403.
2016-08-04	4.9	Updated based on review ID: 10569
2016-08-01	4.8	Updated 6.2.21 and 8.10 section for Mcu_17_CCuconRegUpdate
2016-05-25	4.7	Updated Sections as part of Protected mode support implementation: Section 2.3 : Interface diagram updated Section 3 : File structure updated Section 4.2.17 to 4.2.20 : McuGeneral Configuration parameters added Section 6.2 : IO mode row of APIs updated Section 8.6 : Example usage updated
2016-02-01	4.6	Updated Sections: 4.3.13.53 McuErayPllK3Divider parameter range is updated.
2015-11-12	4.5	Updated Sections: 4.5.1 McuResetReason Configuration for MCU_RESET_MULTIPLE McuPublishedInformation container is removed for AS321
2015-10-15	4.4	Updated Sections: 6.2 (All sub-sections) for I/O privileges, enabling MCU driver to run in User-1 mode also. 4.3.13.12 McuClockSettingMode updated for Range of values.
2015-09-08	4.3	Updated Sections: 6.2.1 Mcu_Init and 7 Error Classification for CLC initialization failure. 4.2.4 Mcu PLL Initialization Delay, 4.2.5 Mcu Eray PLL initialization Delay, 4.3.13.8 McuK2DivRampToPllConfDelay and 4.3.13.9 McuK2DivRampToBackUpConfDelay for SW based delay loops 6.2.1 Mcu_Init, 6.2.3 Mcu_InitClock, 6.2.4 Mcu_DistributePllClock and 6.2.16 Mcu_RampToBackUpClockFreq for caveats
2015-06-15	4.2	Added implementation comments for Mcu_InitClock and Mcu_DistributePllClock.
2015-05-13	4.1	Added user note for McuK2DivSteps.
2015-04-22	4.0	Support added for McuDeInitApi and McuDeInitApi features.
2015-03-10	3.9	Added assumption for TC21x, TC22x Max frequency.
2015-01-15	3.8	Updated for UTP AI00251544:additional API for reset of cold reset status bits in RSTSTAT AI00252697: Configurable ERAY PLL initialization delay after cold power-up AI00252694: Configurable PLL delay for initialization after cold power-up AI00251155: configurable delay for K2 divider step increase/decrease AI00252756: McuResetSetting made multi configuration capable
2014-11-13	3.7	Updated for UTP AI00251090
2014-09-10	3.6	Updated for UTP AI00248867

Date	Version	Change Description
2014-08-12	3.5	Added ADC_DRIVER for Ccu6 module usage and modified in Ccu configuration
2014-07-14	3.4	Removed reference for Software Safety information document
2014-06-16	3.3	Review comments incorporated
2014-06-06	3.2	Support for FM PLL Disabling of ERAY PLL to support 8MHz oscillator frequency
2014-03-26	3.1	Reference to Safety Software information document corrected
2014-02-28	3.0	Added support for EP Platforms
2014-02-04	2.9	Review comments incorporated
2014-01-13	2.8	MCU Standby support
2013-11-11	2.7	Review comments incorporated
2013-11-08	2.6	Corrected EthDivider description; also added traceability tags
2013-09-18	2.5	Fixed review comments
2013-09-12	2.4	Added EruConfiguration container description. Corrected default values for configuration parameters.
2013-08-21	2.3	Update for TC26x, TC29x derivatives as well as safety configuration parameters. Added Chapter Safety information.
2013-04-04	2.2	Fixed review comments
2013-01-04	2.1	Updated Common Published Information
2013-06-02	2.0	Updated for B - step Changes
2012-18-12	1.8	Updated File Structure
2012-26-10	1.7	Added AscLinConfiguration container.
2012-20-10	1.6	AI00066335. CCU6 support
2012-13-08	1.5	Updated AS32 related code.
2012-15-07	1.4	Update Clock Reference Point container. Added DEM Container.
2012-14-06	1.3	Modified clock frequency ratio table.
2012-18-05	1.2	Added Note for Fbbb frequency
2012-12-05	1.1	Added Eray related API and Dividers. Changed default values of Clock frequency and dividers.
2011-20-10	1.0	Allowed clock ratios are updated with respect to TC2Dx device.
2011-04-10	0.3	Added new clock parameters for SPB, FSI, CPU0 and SRI frequencies.
2011-04-08	0.2	Added new clock parameters for SPB, FSI, CPU0, MAX, GTM, SRI and STM frequencies.
2011-20-03	0.1	Initial version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com

Table of Contents **Page**

Table of Contents **Page**

Table of Contents **Page**

Table of Contents **Page**

Table of Contents **Page**

List of Figures **Page**

List of Tables **Page**

List of Tables **Page**

List of Tables **Page**

List of Tables **Page**

List of Code Listings **Page**

1 Introduction

This User Manual provides information regarding the functionality, configuration parameters and API implementation for the MCU driver.

This User Manual is intended to help the user to get acquainted with the MCU driver implementation for the AURIX Hardware Platform. It is the documentation that describes how to use the MCU driver.

1.1 Scope

This document addresses the driver implementation for the following variants of

AURIX HE Family: TC264, TC265, TC275, TC277, TC297, TC298 and TC299 derivatives.

AURIX EP Family: TC233, TC234, TC237, TC222, TC223, TC224, TC212, TC213 and TC214 derivatives

Some of the peripheral units/feature might not be available in all the variants. In such cases, the relevant information should be discarded for the variant where the unit/feature is not applicable.

This document addresses the following features of the MCU driver implementation,

- AURIX microcontroller HW modules used to realize the MCU driver
- File structure of the MCU driver
- Configuration parameters of the MCU driver
- Parameters published by the MCU driver
- API Specification
- Interrupt Handling
- Common use cases with code snippets.
- Safety information
- Limitations and Assumptions

This documentation applies to the MCU driver Variant: Variant PB.

1.2 Abbreviations

The abbreviations used inside this document are explained in **Table 1**.

Table 1 Abbreviations used in this document

Abbreviation	Explanation
“MCC XLS” or “MCC”	**Aurix_MC-ISAR_MCU_Clock_Calculator.xlsm** sheet delivered along with Mcu package. This helps the user by computing the clock parameters in a simple and friendly way.
µC	Microcontroller
µs	Microsecond
API	Application Programming Interface
AURIX	Automotive Realtime Integrated NeXt Generation Architecture
AURIX EP	AURIX Efficiency Performance family
AURIX HE	AURIX High End microcontroller family
AUTOSAR	AUTomotive Open System Architecture
CC	Capture/Compare
CCU	Clock Control Unit
CCU6	Capture/Compare Unit 6
CCU6_PISEL0	CCU6 Port Input Select register 0
CCU6_TCTR0	CCU6 Timer Control Register 0

CGU	Clock Generation Unit
CPU	Central Processing Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EBU	External Bus Unit
ECU	Electronic Control Unit
ERAY	FlexRay
ESR0/ESR1	External System Request 0/ External System Request 1 triggers
FM PLL	Frequency Modulated Phase Lock Loop
HW	Hardware
IFX	Infineon Technologies
MCAL	MicroController Abstraction Layer
MC-ISAR	Microcontroller Infineon Software Architecture
MCU	MicroController Unit
NMI	Non Maskable Interrupt
OSC	Oscillator
PLL	Phase Lock Loop
PORST	Power On Reset
RAM	Random Access Memory
ROM	Read Only Memory
SCU	System Control Unit
SCU_ARSTDIS	SCU Application Reset Disable Register
SCU_CCUCON<x>	SCU Clock Control Unit Control register <x>
SCU_EICR<x>	SCU External Input Channel Register <x>
SCU_IGCR<x>	SCU Interrupt Gating Control Register <x>
SCU_PLLERAYCON<x>	SCU PLL ERAY Configuration register <x>
SCU_PLLCON<x>	SCU PLL Configuration register <x>
SCU_PMCSR<x>	SCU Power Management Control and Status Register <x>
SCU_PMSWCR<x>	SCU Power Management Standby and Wakeup Control Register <x>
SCU_RSTCON	SCU Reset Configuration register
SCU_SWRSTCON	SCU SW Reset Configuration register
SMU	Safety Management Unit
STM	System Timer
SW	Software
VCO	Voltage Controlled Oscillator
WDT	Watch Dog Timer

1.3 References

This section lists down all relevant documents for this User Manual. Some of the artifacts listed below will not contain the version number. For those artifacts, kindly refer the latest version

[1] Requirements on MCU

AUTOSAR_SRS_MCU_Driver.pdf

[2] Specification of MCU

AUTOSAR_SWS_MCU_Driver.pdf (Rel 4.0.3)

[3] Target TC27x User Manual: tc27x_um_V1.3.1.pdf

[4] Aurix_MC-ISAR_MCU_Clock_Calculator.xlsm

[5] Aurix_MC-ISAR_UM_GTMDriver.pdf

[6] Target TC26x User Manual: tc26x_um_V1.0.pdf

[7] Target TC29x User Manual: tc29x_um_V1.1.pdf

[8] Target TC23x/TC22x/TC21x User Manual: tc23x_tc22x_um_v1.0.pdf

[9] SYSPLL Frequency modulation Application Note v1.1: AP3224411_AURIX_SYSPLL_FM.pdf

[10] MC-ISAR_AURIX_UM_MCALLib.pdf

2 Driver Overview

The MCU driver provides services for

- Initialization of MCU clock/PLL's, clock prescalers and MCU clock distribution
- Initialization of RAM sections
- Activation of microcontroller power modes
- Activation of a microcontroller reset
- Provides a service to get the reset reason from hardware

MCU driver accesses the microcontroller hardware directly and is located in the Microcontroller Abstraction layer (MCAL).

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The **Figure 1** illustrates the Hardware peripherals and their interaction with the Software drivers.

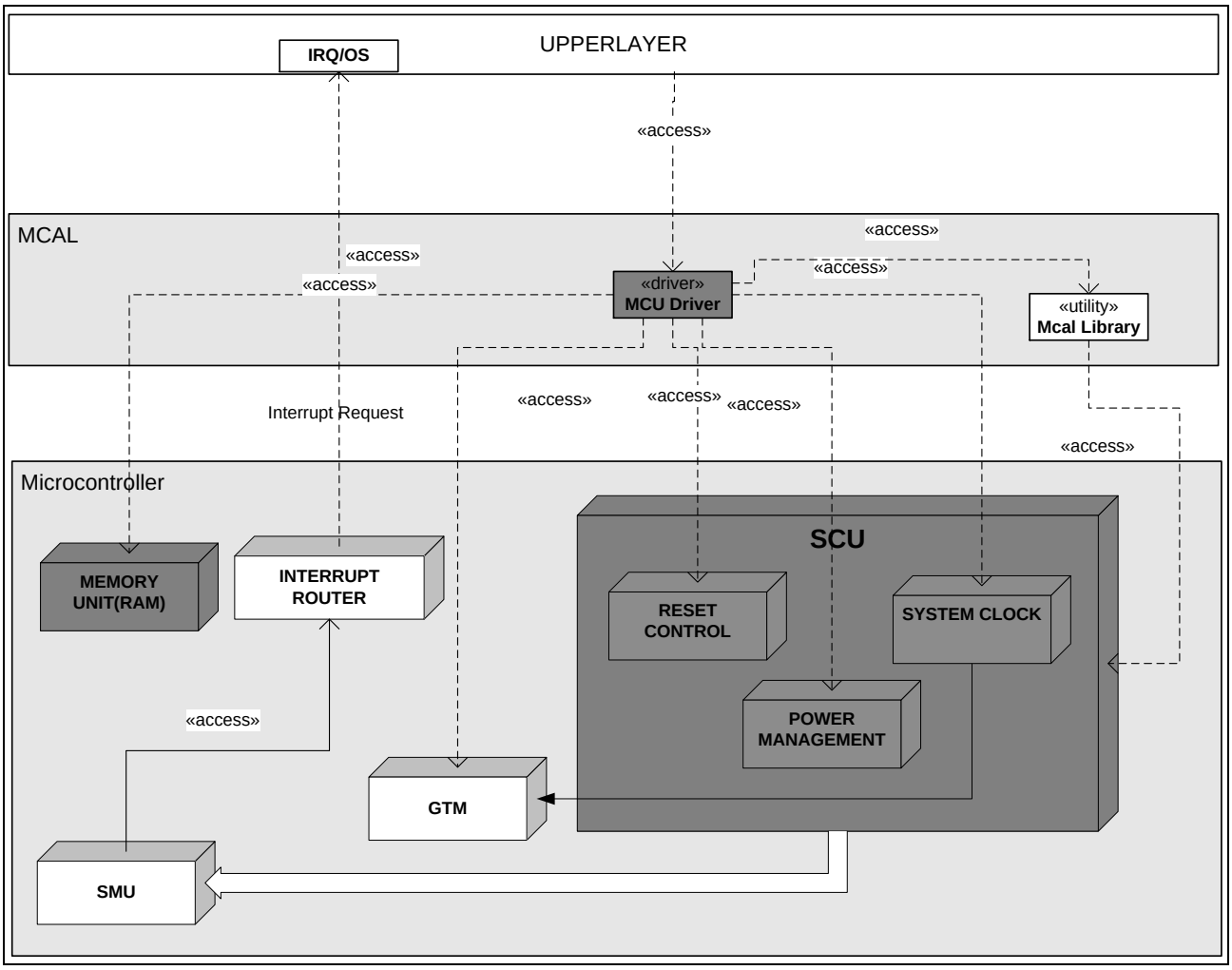

Figure 1 SW HW Mapping

2.2 Hardware Features

2.2.1 Memory Unit

Memory unit as shown in the **Figure 1** depicts the MCU driver ownership. It initializes the RAM section, whose base address, RAM section size (in bytes) and data to be initialized is configured by user in the configuration files.

RAM section is initialized for the specified memory section size, from the specified memory address. RAM section size to be initialized and the start address are configurable. The data to be initialized is also configured by the user. The verification of this RAM section is not responsibility of the MCU driver.

Table 2 RAM Memory ranges for AURIX HE and EP devices

Name	Start Address	End Address
LMU (Not available in TC26x, TC23x, TC22x and TC21x)	0x90000000	0x90007FFF
CPU0 DSPR	0x70000000	0x70011FFF (TC26x)
		0x7001BFFF (TC27x)
		0x7001DFFF (TC29x)
		0x7002DFFF (TC23x)
		0x70015FFF (TC22x/TC21x)
CPU0 PSPR	0x70100000	0x70103FFF (TC26x)
		0x70105FFF (TC27x)
		0x70107FFF (TC29x)
		0x70101FFF (TC22x/TC21x/TC23x)
CPU1 DSPR (Not available in AURIX EP family)	0x60000000	0x6001DFFF (TC26x)
		0x6001DFFF (TC27x)
		0x6003BFFF (TC29x)
CPU1 PSPR (Not available in AURIX EP family)	0x60100000	0x60107FFF
CPU2 DSPR (Not available in TC26x and AURIX EP family)	0x50000000	0x5001DFFF (TC27x)
		0x5003BFFF (TC29x)
CPU2 PSPR (Not available in TC26x and AURIX EP family)	0x50100000	0x50107FFF

2.2.2 System Clock

The Clock Generation Unit in the AURIX HE and TC23x family consists of an oscillator circuit and two Phase-Locked Loop modules (PLL and PLL_ERAY) and a Clock Control Unit. The PLL_ERAY unit does not

exist in TC22x and TC21x AURIX EP derivatives. The PLL_ERAY block gets disabled by putting the ERAY PLL block in power saving mode when Oscillator frequency is below 16 MHz.

MCU driver supports PLL mode only; it doesn't support Clock Generation Unit to run on backup clock.

MCU driver supports two modes of system clock generation:

2.2.2.1 Normal Mode

In Normal Mode, the PLL is running at the external frequency fOSC and fPLL is fOSC divided down by a factor P, multiplied by a factor N and then divided down by a factor K2.

The PLL is further divided by the factors in the clock control unit to arrive at different clocks such as SRI, FSI, FSI2, CPUx, SPB, MAX, ETH, STM, BBB, CAN, ERAY, BAUD1, BAUD2, ASCLIN, EBU (present only in TC29x) and GTM. CPU2 clock is not present in TC26x variant. CPU2, CPU1 clocks are not present in AURIX EP platforms. BAUD1 clock is not present in TC23x/TC22x/TC21x EP platforms.

Clock Initialization is done in steps (through Mcu_InitClock API) in accordance with target user manual. There are four dividers available in the configuration (P, N, K1, K3 and K2). P, N, K2 and K3 are required to achieve the desired PLL frequency. The supplied K2 value is used to achieve the final target frequency. The number of steps to be done is driven by the configuration parameter K2-Divider Steps (**Refer 4.3.13.7**). K2-divider Steps parameter denotes only the intermediate steps and not the complete set, in other words, minimum of two steps will be always done. For instance, if the K2-Divider Steps value is supplied with value "x", the actual number performed steps will be "x + 2". The range of "x" is between 0 and 6.

2.2.2.2 Pre-scalar Mode

In Pre-scalar Mode, the PLL is running at the external frequency fOSC and fPLL is derived from fOSC only by the K1 divider.

2.2.2.3 Clock Calculation

The formula used to set the Clock is given by the equation

For Normal Mode $f_{pll} = \frac{N}{P \times K2} f_{osc}$

For Normal Mode $f_{pll2} = \frac{N}{P \times K3} f_{osc}$

Care should be taken to ensure *minimum range allowed* <= f_{pll} <=*maximum range allowed* (in MHz)

For Prescaler Mode

$$f_{pll} = \frac{f_{osc}}{K1}$$

Where N = Ndiv + 1,
P = Pdiv + 1,
K1 = K1div + 1,
K2 = K2div + 1,
K3 = K3div + 1,

The frequency range for AURIX HE and EP devices are mentioned in the **Table 3** below.

Table 3 Frequency ranges for AURIX HE and EP device

Name	Minimum limit (MHz)	Maximum limit (MHz)
Fosc	8	40

Fref	8	24
Fpll	20	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
		133 (TC21x/TC22x)
Fvco	400	800
Fref (Eray)	16	24
Fpll (Eray)	20	400
Fvco (Eray)	400	480
Fsri	-	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
		133 (TC21x/TC22x)
Fmax	-	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
		133 (TC21x/TC22x)
Fcpu0	-	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
		133 (TC21x/TC22x)
Fcpu1 (Not available in AURIX EP family)	-	200 (TC26x/TC27x)
		300 (TC29x)
Fcpu2 (Not available in AURIX EP family)	-	200 (TC27x)
		300 (TC29x)
Fspb	-	100
		66.5 (TC21x/TC22x)
Fasclinf	-	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
		133 (TC21x/TC22x)
Fasclins	-	100
		66.5 (TC21x/TC22x)
Fbaud1 (Not available in TC23x, TC22x and TC21x)	-	100
Fbaud2	-	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
		133 (TC21x/TC22x)
Ffsi	-	100
		66.5 (TC21x/TC22x)
Ffsi2	-	200 (TC26x/TC27x/TC23x)
		300 (TC29x)
Fstm	-	100
		66.5 (TC21x/TC22x)

Fgtm	-	100
		66.5 (TC21x/TC22x)
Feray (Not available in TC22x and TC21x)	-	80
Fbbb	-	100 (TC26x/TC27x/TC23x)
		150 (TC29x)
		66.5 (TC21x/TC22x)
Fcan	-	100
		66.5 (TC21x/TC22x)
Fvadc	-	133 (TC27x/TC29x)
		80 (TC26x)
		Same as Fspb (TC22x/TC21x/TC23x)

The allowed clock ratios for AURIX HE are available in **Table 4**.

The equations for different clock parameters are given below.

$$f_{SRI} = \frac{f_{PLL}}{SRIDIV}$$

$$f_{FSI} = \frac{f_{SRI}}{FSIDIV}$$

$$f_{FSI2} = \frac{f_{SRI}}{FSI2DIV}$$

$$f_{CPU0} = \frac{f_{SRI} * (64 - CPU0DIV)}{64}$$

$$f_{CPU1} = \frac{f_{SRI} * (64 - CPU1DIV)}{64}$$ (Not present in AURIX EP family)

$$f_{CPU2} = \frac{f_{SRI} * (64 - CPU2DIV)}{64}$$ (Not present in TC26x and AURIX EP family)

$$f_{SPB} = \frac{f_{PLL}}{SPBDIV}$$

$$f_{MAX} = \frac{f_{PLL}}{MAXDIV}$$

$$f_{STM} = \frac{f_{PLL}}{STMDIV}$$

$$f_{GTM} = \frac{f_{PLL}}{GTMDIV}$$

$$F_{CAN} = \frac{f_{PLL}}{CANDIV}$$

$F_{ASCLINF} = \frac{f_{PLL}}{ASCLINFDIV}$

$F_{ASCLINS} = \frac{f_{PLL}}{ASCLINSDIV}$

$F_{BAUD1} = \frac{f_{PLL}}{BAUD1DIV}$ (Not present in TC23x/TC22x/TC21x)

$F_{BAUD2} = \frac{f_{PLL}}{BAUD2DIV}$

$F_{EBU} = \frac{f_{PLLERAY}}{EBUDIV}$ (Present only in TC29x)

Refer the **Table 4** for more information on clock ratios

2.2.3 Allowed clock ratios

The dividers are to be carefully selected so as to satisfy the below given conditions.

Table 4 Allowed clock ratios

CLOCK A	CLOCK B	Default Configured Ratio	Allowed ratios	Notes
f_{MAX}	--	--	f_{MAX} is always equal to the fastest clock derived out of f_{SOURCE}	--
f_{SRI}	f_{SPB}	1:2	1:n	n=1,2,3,4
f_{SRI}	f_{FSI}	1:2	1:n	n=1,2 (For TC26x/TC27x/TC23x/TC22x/TC21x) n=1,2,3 (For TC29x)
f_{SRI}	f_{FSI2}	1:2	1:n	n=1,2 (For TC26x/TC27x/TC23x/TC22x/TC21x) n=1,2,3 (For TC29x)
f_{FSI2}	f_{FSI}	1:1	1:n	n=1,2 (For TC26x/TC27x/TC23x/TC22x/TC21x) n=1,2,3 (For TC29x)
f_{GTM}	f_{SPB}	1:1	1:n[1]	n=1,2,3,4,5…
f_{SPB}	f_{GTM}	1:1	1:n[1]	n=1,2,3,4,5…
f_{STM}	f_{SPB}	1:1	1:n[2]	n=1,2,3,4,5…
f_{SPB}	f_{STM}	1:1	1:n[2]	n=1,2,3,4,5…
f_{SRI}	f_{BBB}[3]	1:1	1:n	n=1,2
f_{BAUD1}	f_{SPB}	1:1	1:n[4]	n=1,2,3,4,5…

f_{SPB}	f_{BAUD1}	1:1	1:n[4]	n=1,2,3,4,5…
f_{BAUD2}	f_{SPB}	1:1	1:n[5]	n=1,2,3,4,5…
f_{SPB}	f_{BAUD2}	1:1	1:n[5]	n=1,2,3,4,5…
f_{CAN}	f_{SPB}	1:1	1:n[6]	n=1,2,3,4,5…
f_{SPB}	f_{CAN}	1:1	1:n[6]	n=1,2,3,4,5…
$f_{ASCLINF}$	f_{SPB}	1:1	1:n[7]	n=1,2,3,4,5…
f_{SPB}	$f_{ASCLINF}$	1:1	1:n[7]	n=1,2,3,4,5…
$f_{ASCLINS}$	f_{SPB}	1:1	1:n[8]	n=1,2,3,4,5…
f_{SPB}	$f_{ASCLINS}$	1:1	1:n[8]	n=1,2,3,4,5…
f_{EBU}	f_{SRI}	--	1:n	n=1,2,3,4,5…(For TC29x only)

Note:

1. fGTM can be faster, slower, or equal to fSPB
2. fSTM can be faster, slower, or equal to fSPB
3. fBBB can be faster or equal to fSPB
4. fBAUD1 can be faster, slower, or equal to fSPB (Not present in TC23x/TC21x/TC22x)
5. fBAUD2 can be faster, slower, or equal to fSPB
6. fCAN can be faster, slower, or equal to fSPB
7. fASCLINF can be faster, slower, or equal to fSPB
8. fASCLINS can be faster, slower, or equal to fSPB

For AURIX HE family:

With the default configuration, $f_{PLL} = f_{SRI} = f_{Max} = 160MHz$.

And, $f_{CPU0} = f_{CPU1} = f_{CPU2} = f_{FSI} = f_{FSI2} = f_{SPB} = f_{BBB} = f_{GTM} = f_{STM} = f_{CAN} = f_{BAUD1} = f_{BAUD2} = f_{ASCLINF} = 80MHz$.

To achieve a CPUx clock speed of 160MHz do the following change:

- The value of McuCPUxDivider has to be changed to 0. Then, $f_{CPUx} = f_{SRI} = 160MHz$.

For AURIX EP family:

With the default configuration, $f_{PLL} = f_{SRI} = f_{Max} = 120MHz$.

And, $f_{CPU0} = f_{CPU1} = f_{CPU2} = f_{FSI} = f_{FSI2} = f_{SPB} = f_{BBB} = f_{GTM} = f_{STM} = f_{CAN} = f_{BAUD1} = f_{BAUD2} = f_{ASCLINF} = 60MHz$.

To achieve a CPUx clock speed of 120MHz do the following change:

The value of McuCPUxDivider has to be changed to 0. Then, $f_{CPUx} = f_{SRI} = 120MHz$.

Refer**[3], [6], [7]** for more options on clock frequencies and dividers. It gives multiple options for different clocks for a given PLL frequency. The clock dividers and frequencies generated by **[4]** are validated for all clock boundaries and ratios.

2.2.4 Power management

There are three power-management modes: Run Mode, Idle Mode and Sleep Mode. These modes are summarized in the following table.

Table 5 Power management summary

Mode	Description
Run	The system is fully operational. CPU and peripherals are enabled as determined by software.
Idle	The CPU clock is disabled, waiting for a condition to return it to Run Mode. Idle Mode can be entered by software when the processor has no active tasks to perform, or it can be entered automatically if the SMU is configured to stop a CPU on detection of a safety alarm. The peripherals remain active clocked. Processor memory is accessible to peripherals. An Application Reset will return the CPU to Run Mode. If the idle mode was requested by SW, then a CPU Watchdog Timer event, an SMU event, an NMI trap or any interrupt for the CPU will also return the system to Run Mode.
Sleep	All the CPU(s) are set into idle mode. Only those peripherals programmed to operate in Sleep Mode are active. The Ports will retain their programmed state. The other peripheral modules will be shut down by the sleep signal. Interrupts for any CPU or master CPU, any SMU event, an NMI trap, or an Application or System Reset will return the system to Run Mode. Entering this state requires an orderly shutdown controlled by the Power Management State Machine.
Standby	Standby Mode can be entered by software when the master CPU(s) issues a system standby request. The power to the main 1.3 V core and 3.3 V domains are switched off and only the 1.3 V Standby domains are actively supplied. The reset/NMI/wake-up logic (PORST/ESR1/PinA/PinB wakeup pins) pads/ and the standby RAMs are kept alive. The wake-up is triggered when an edge event is detected at the configured ESR1/PinA/PinB wakeup pins.

The system will return to Run Mode from SLEEP/IDLE mode through occurrence of any of the following conditions:

- An interrupt is received from an interrupt source of the CPU
- An NMI trap request is received
- An Application Reset is generated

The system will do a warm reset when woken up from STANDBY mode through the configured wakeup pins.
Please refer “Power Management” section in the respective UM **[3], [6], [7]** for more details.

2.2.5 Reset Control

The following reset triggers are available:

- External power-on hardware reset request trigger ($\overline{POSRT}$)(cold reset)
- External System Reset request triggers ($\overline{ESR0}$, $\overline{ESR1}$) (warm reset)
- Safety Management Unit (SMU) reset request (warm reset)
- Software reset (warm reset)
- STM reset (STM0, STM1, and STM2).
- Cerberus reset (CB0, CB1 and CB3).
- Tuning Protection reset (TP reset)
- Supply Watchdog reset. (SWD)
- Standby regulator watchdog reset (STBYR)
- Debug(OCDS) reset request trigger (warm reset)
- Resets via the JTAG interface

- JTAG reset (special reset).

2.3 Software Driver Description

The figure shows the Application Programming Interface (API) for the MCU driver and its interaction with other software drivers.

− Development Error Tracer (DET) is required if DET is enabled for the MCU driver.

− Diagnostic Event Manager (DEM) can be enabled for MCU driver

− Safety reporting will be enabled if safety feature is enabled

The APIs of the driver are explained in detail in chapter **6.1.15**

The configuration data of the driver is also depicted in the figure. The individual containers and their configuration parameters are explained in detail in chapter **4**.

Table 6 Configuration Parameters – Difference between Aurix derivatives

Configuration Parameter	Remarks
McuSystemModeCpuCore, McuIdleModeCpuCore, McuStm1ResetDisable, McuStm2ResetDisable, STM1ResetTrigger, STM2ResetTrigger, McuCPU1Divider, McuClockCPU1Frequency, McuCPU2Divider, McuClockCPU2Frequency, McuEBUDivider, McuClockEBUFrequency, ASCLIN2, ASCLIN3	These configuration parameters are not applicable for AURIX EP platforms.
McuBAUD1Divider, McuClockBAUD1Frequency	This configuration parameter is not applicable for TC23x\TC22x\TC21x
McuClockSelectADC	This configuration parameter is not applicable for TC23x\TC22x\TC21x
McuErayPllPDivider, McuErayPllNDivider, McuErayPllK2Divider, McuErayPllK3Divider, McuClockErayPllFrequency, McuClockErayPll2Frequency, McuErayDivider, McuClockErayFrequency	This configuration parameter is not applicable for TC22x\TC21x
McuETHDivider, McuETHFrequency	This configuration parameter is not applicable for TC23x without ADAS\TC22x\TC21x
DmaChannel multiplicity	64 channels – TC27x 48 channels – TC26x 128 channels – TC29x 16 channels – TC21x\TC22x\TC23x 24 channels – TC24x

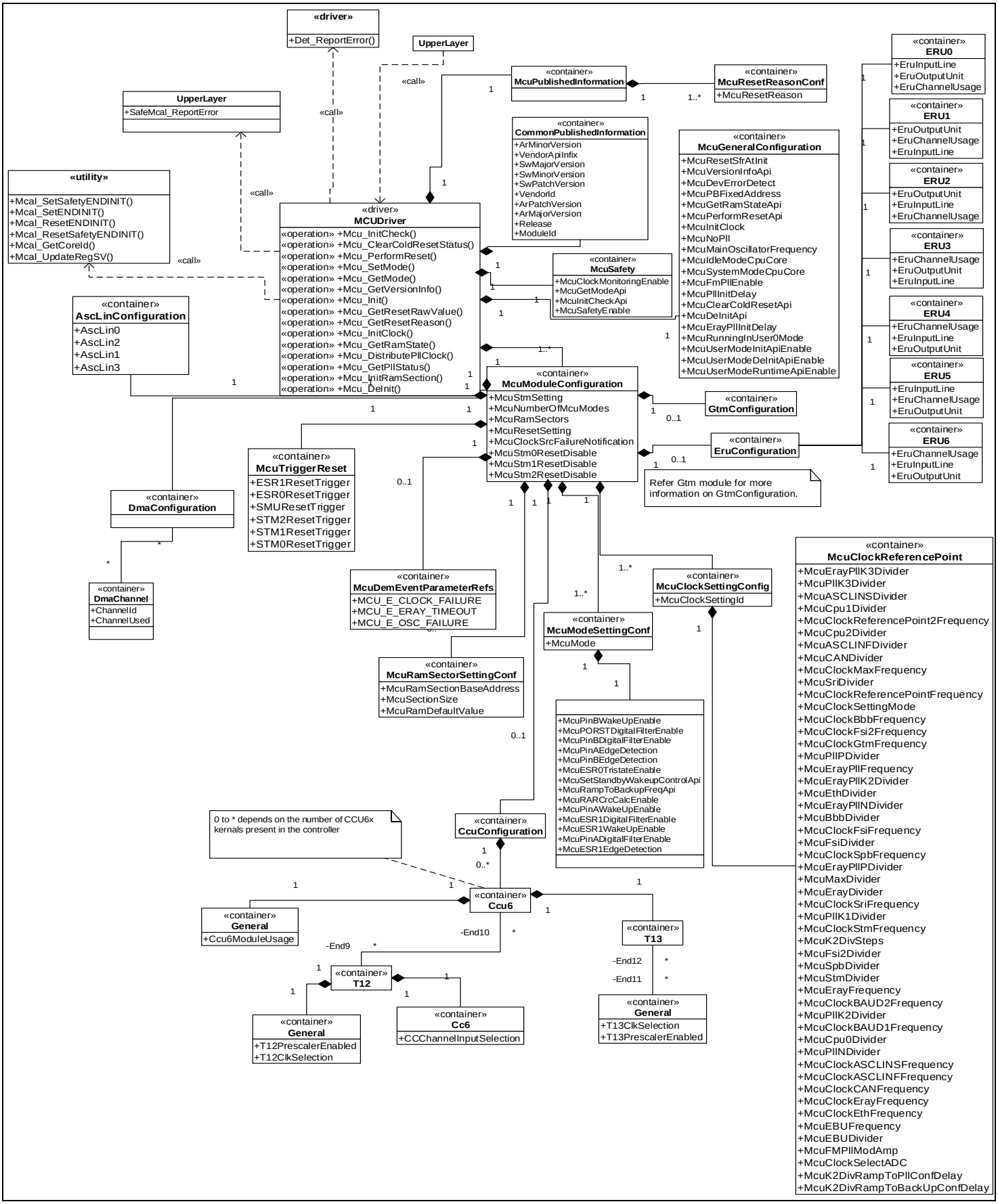
«driver»
+Det_ReportError()
UpperLayer
«call»
UpperLayer
+SafeMcal_ReportError
«container»
McuPublishedInformation
«container»
McuResetReasonConf
+McuResetReason
«container»
ERU0
+EruInputLine
+EruOutputUnit
+EruChannelUsage
«container»
ERU1
+EruOutputUnit
+EruChannelUsage
+EruInputLine
«container»
ERU2
+EruOutputUnit
+EruInputLine
+EruChannelUsage
«container»
ERU3
+EruChannelUsage
+EruOutputUnit
+EruInputLine
«container»
ERU4
+EruChannelUsage
+EruInputLine
+EruOutputUnit
«container»
ERU5
+EruInputLine
+EruChannelUsage
+EruOutputUnit
«container»
ERU6
+EruChannelUsage
+EruInputLine
+EruOutputUnit
«container»
CommonPublishedInformation
+ArMinorVersion
+VendorApiInfix
+SwMajorVersion
+SwMinorVersion
+SwPatchVersion
+VendorId
+ArPatchVersion
+ArMajorVersion
+Release
+ModuleId
«container»
McuGeneralConfiguration
+McuResetSfrAtInit
+McuVersionInfoApi
+McuDevErrorDetect
+McuPBFixedAddress
+McuGetRamStateApi
+McuPerformResetApi
+McuInitClock
+McuNoPll
+McuMainOscillatorFrequency
+McuIdleModeCpuCore
+McuSystemModeCpuCore
+McuFmPllEnable
+McuPllInitDelay
+McuClearColdResetApi
+McuDeInitApi
+McuErayPllInitDelay
+McuRunningInUser0Mode
+McuUserModeInitApiEnable
+McuUserModeDeInitApiEnable
+McuUserModeRuntimeApiEnable
«utility»
+Mcal_SetSafetyENDINIT()
+Mcal_SetENDINIT()
+Mcal_ResetENDINIT()
+Mcal_ResetSafetyENDINIT()
+Mcal_GetCoreId()
+Mcal_UpdateRegSV()
«call»
«driver»
MCUDriver
«operation» +Mcu_InitCheck()
«operation» +Mcu_ClearColdResetStatus()
«operation» +Mcu_PerformReset()
«operation» +Mcu_SetMode()
«operation» +Mcu_GetMode()
«operation» +Mcu_GetVersionInfo()
«operation» +Mcu_Init()
«operation» +Mcu_GetResetRawValue()
«operation» +Mcu_GetResetReason()
«operation» +Mcu_InitClock()
«operation» +Mcu_GetRamState()
«operation» +Mcu_DistributePllClock()
«operation» +Mcu_GetPllStatus()
«operation» +Mcu_InitRamSection()
«operation» +Mcu_DeInit()
«container»
McuSafety
+McuClockMonitoringEnable
+McuGetModeApi
+McuInitCheckApi
+McuSafetyEnable
«container»
AscLinConfiguration
+AscLin0
+AscLin2
+AscLin1
+AscLin3
«container»
McuModuleConfiguration
+McuStmSetting
+McuNumberOfMcuModes
+McuRamSectors
+McuResetSetting
+McuClockSrcFailureNotification
+McuStm0ResetDisable
+McuStm1ResetDisable
+McuStm2ResetDisable
«container»
GtmConfiguration
«container»
EruConfiguration
Refer Gtm module for more information on GtmConfiguration.
«container»
McuTriggerReset
+ESR1ResetTrigger
+ESR0ResetTrigger
+SMUResetTrigger
+STM2ResetTrigger
+STM1ResetTrigger
+STM0ResetTrigger
«container»
DmaConfiguration
«container»
DmaChannel
+ChannelId
+ChannelUsed
«container»
McuDemEventParameterRefs
+MCU_E_CLOCK_FAILURE
+MCU_E_ERAY_TIMEOUT
+MCU_E_OSC_FAILURE
«container»
McuClockSettingConfig
+McuClockSettingId
«container»
McuModeSettingConf
+McuMode
«container»
McuRamSectorSettingConf
+McuRamSectionBaseAddress
+McuSectionSize
+McuRamDefaultValue
+McuPinBWakeUpEnable
+McuPORSTDigitalFilterEnable
+McuPinBDigitalFilterEnable
+McuPinAEdgeDetection
+McuPinBEdgeDetection
+McuESR0TristateEnable
+McuSetStandbyWakeupControlApi
+McuRampToBackupFreqApi
+McuRARCrcCalcEnable
+McuPinAWakeUpEnable
+McuESR1DigitalFilterEnable
+McuESR1WakeUpEnable
+McuPinADigitalFilterEnable
+McuESR1EdgeDetection
0 to * depends on the number of CCU6x kernals present in the controller
«container»
CcuConfiguration
«container»
Ccu6
«container»
General
+Ccu6ModuleUsage
«container»
T13
«container»
T12
«container»
General
+T12PrescalerEnabled
+T12ClkSelection
«container»
Cc6
+CCChannelInputSelection
«container»
General
+T13ClkSelection
+T13PrescalerEnabled
-End9
-End10
-End12
-End11
«container»
McuClockReferencePoint
+McuErayPllK3Divider
+McuPllK3Divider
+McuASCLINSDivider
+McuCpu1Divider
+McuClockReferencePoint2Frequency
+McuCpu2Divider
+McuASCLINFDivider
+McuCANDivider
+McuClockMaxFrequency
+McuSriDivider
+McuClockReferencePointFrequency
+McuClockSettingMode
+McuClockBbbFrequency
+McuClockFsi2Frequency
+McuClockGtmFrequency
+McuPllPDivider
+McuErayPllFrequency
+McuErayPllK2Divider
+McuEthDivider
+McuErayPllNDivider
+McuBbbDivider
+McuClockFsiFrequency
+McuFsiDivider
+McuClockSpbFrequency
+McuErayPllPDivider
+McuMaxDivider
+McuErayDivider
+McuClockSriFrequency
+McuPllK1Divider
+McuClockStmFrequency
+McuK2DivSteps
+McuFsi2Divider
+McuSpbDivider
+McuStmDivider
+McuErayFrequency
+McuClockBAUD2Frequency
+McuPllK2Divider
+McuClockBAUD1Frequency
+McuCpu0Divider
+McuPllNDivider
+McuClockASCLINSFrequency
+McuClockASCLINFFrequency
+McuClockCANFrequency
+McuClockErayFrequency
+McuClockEthFrequency
+McuEBUFrequency
+McuEBUDivider
+McuFMPllModAmp
+McuClockSelectADC
+McuK2DivRampToPllConfDelay
+McuK2DivRampToBackUpConfDelay

Figure 2 Driver Description for AUTOSAR 4.0.x

3 File structure

This section provides details about the MCU driver files and other related files.

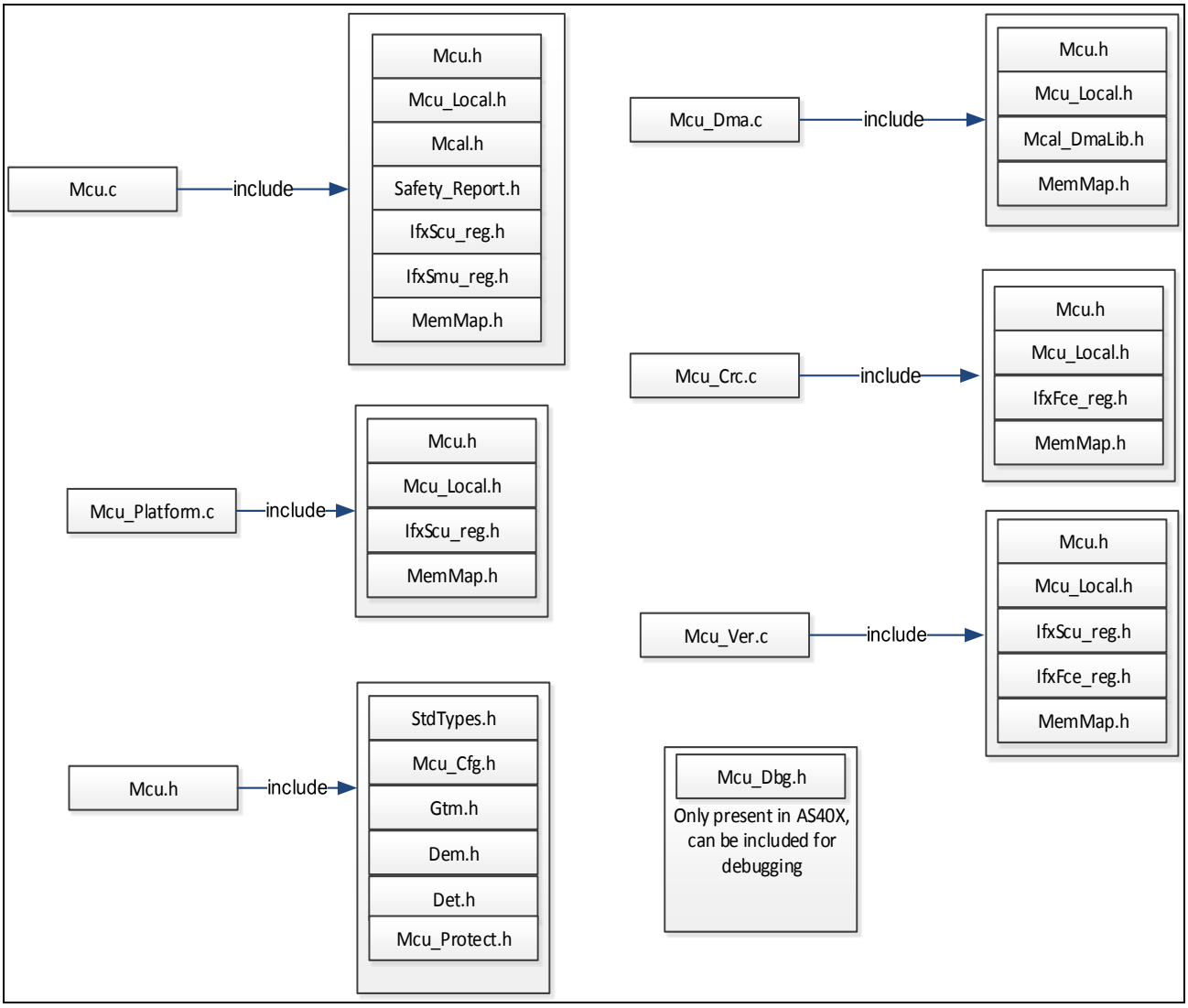

Figure 3 Driver File Structure

Table 7 File Contents

File Name	File Contents
Mcu.h	This header file exports macros, type definitions and function prototypes for the MCU driver
Mcu_Local.h	This header file contains local macros, type definitions and function prototypes for the MCU Driver; which are not exposed to the upper layers.
Mcu.c	This file contains common functionality of the MCU Driver; no specific code with respect to AUTOSAR version or target derivative.
Mcu_Ver.c	This file contains AUTOSAR version specific functionality.
Mcu_Platform.c	This file contains derivative specific functionality.
Mcu_Crc.c	This file contains CRC HW specific initialization functionality. (Not used for

	TC23x/TC22x/TC21x build since FCE HW is not present)
Mcu_Dma.c	This file contains DMA HW specific initialization functionality.
Mcu_Cfg.h	Configuration data of the MCU driver is declared here.
Mcu_PBCfg.c	Configuration data of the MCU driver is defined here.
Gtm.h	This header file exports macros, type definitions and function prototypes for the Gtm driver
IfxScu_reg.h	Includes the register definition file for AURIX microcontroller
IfxSmu_reg.h	Includes the register definition file for SMU
IfxFce_reg.h	Includes the register definition file for FCE. (Not used for TC23x/TC22x/TC21x build since FCE HW is not present)
Mcal.h	Provides library functions for MCAL layer
Mcal_DmaLib.h	Provides definitions to access DMA registers and macros
Mcu_Dbg.h	Provides extern declarations for debug related variables. Only present in AUTOSAR 4.0.x
Mcu_Protect.h	Includes all the first level indirection macros for protected mode support
Safety_Report.h	Header file for reporting safety related errors.
Common Library Files (inclusion by Mcal.h)	
Mcal_TcLib.h	Header file exporting MCAL Tricore specific library functions
Mcal_WdgLib.h	Header file exporting MCAL Watchdog specific library functions
Mcal_Compiler.h	Header file exporting MCAL compiler specific functions
Common AUTOSAR Files (inclusion by MCU header files)	The common AUTOSAR files are provided by AUTOSAR and are used as it is without any changes.
Platform_Types.h	Platform dependencies (AURIX in our case) are declared here.
Compiler.h	Compiler dependencies are declared here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code.
Std_Types.h	Standard data types to be used are declared here.
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections
Det.h	Provides the functionality of Development Error Tracer
Dem.h	Provides the functionality of Diagnostic Event Manager

4 Configuration Documentation

This chapter summarizes
Configuration concept used for structuring the driver
Configuration parameter variance w.r.t supported HW
All configuration parameters and their description **Configuration Concept**

4.1.1 Configuration Class

The development of Basic Software drivers involves the following development cycles:
Compiling, Linking and Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compile configuration requires the driver to be compiled, linked and located and hence the values of pre-compile parameters must be **frozen before compilation** and therefore can be used in source code distributions of the driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to driver at runtime. Since configuration location is statically independent of the driver functionality itself but linked dynamically (e.g. at driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the driver, a driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary**.**

The MCU driver is delivered as a Variant PB.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Topics **4.1.1**, **4.1.2** should be read for understanding the configuration class field

Table 8 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, Variant PC.

Table 9 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

4.2 Container: MCU General configuration

This container contains the configuration (parameters) of the MCU driver.

4.2.1 Development error detection

Table 10 Development Error Tracer configuration

Syntax:	McuDevErrorDetect		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	Values: TRUE - Development error detection enabled FALSE - Development error detection disabled Default value: FALSE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the development error detection and reporting. #define MCU_DEV_ERROR_DETECT (STD_OFF/ON)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.2 Module Version Information

Table 11 Version information API configuration

Syntax:	McuVersionInfoApi		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	Values: TRUE – Version info API enabled FALSE – Version Info API disabled Default value: FALSE		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable / disable the API to read out the driver's version information. #define MCU_VERSION_INFO_API (STD_OFF/ON)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.3 Perform Reset Enable/Disable

Table 12 Perform Reset Enable/Disable

Syntax:	McuPerformResetApi		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	Values: TRUE – Enabled :The function Mcu_PerformReset() is available FALSE – Enabled :The function Mcu_PerformReset() not is available Default value: FALSE		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable / disable the use of the function Mcu_PerformReset (). #define MCU_PERFORM_RESET_API (STD_OFF/ON)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.4 Mcu PLL Initialization Delay

Table 13 McuPllInitDelay

Name:	McuPllInitDelay		
Source:	This is IFX specific parameter.		
File:	Mcu_Cfg.h		
Range:	Values: 5 us .100us Default Value: 10us		
Configuration class	Pre-compile	x	VariantPC
	Link Time	--	--
	Post build	--	--
Description:	This parameter denotes the delay configured for PLL initialization. The delay generated by following macro depends on backup frequency. (i.e. 100 MHz). #define MCU_CONF_DELAY_PLL (<value>) ***Note: The configured delay value is used to generate delay loops based out of software. The delay values may not be accurate due to compiler optimizations / CPU performance and could give a delay time greater than the configured value. It is recommended to fine tune this value based on the same to get accurate delays*. (Refer EPN PLL_TC.005)**		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.5 Mcu Eray PLL initialization Delay

Table 14 McuErayPllInitDelay

Name:	McuErayPllInitDelay		
Source:	This is IFX specific parameter.		
File:	Mcu_Cfg.h		
Range:	Values: 5us ..100us Default Value: 10us		
Configuration class	Pre-compile	X	VariantPC
	Link Time	--	--
	Post build	--	--
Description:	This parameter denotes the delay configured for PLL initialization. The delay generated by following macro depends on backup frequency. (i.e. 100 MHz). #define MCU_CONF_DELAY_ERAY_PLL (<value>) ***Note: The configured delay value is used to generate delay loops based out of software. The delay values may not be accurate due to compiler optimizations / CPU performance and could give a delay time greater than the configured value. It is recommended to fine tune this value based on the same to get accurate delays*.(Refer EPN : PLL_ERAY_TC.001)**		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.6 Mcu Oscillator Frequency

Table 15 McuMainOscillatorFrequency

Name:	McuMainOscillatorFrequency		
Source:	This is IFX specific parameter.		
File:	Mcu_Cfg.h		
Range:	Range: 8MHz <= McuMainOscillatorFrequency<= 40MHz Default : 20 MHz		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter denotes External crystal frequency value in MHz #define MCU_MAIN_OSC_FREQ (<value>) If McuMainOscillatorFrequency < 16 MHz, ERAY PLL block is disabled since it cannot operate below 16MHz oscillator frequency.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.7 Mcu PB Fixed Address

Table 16 PostBuild Fixed Address Feature

Syntax:	McuPBFixedAddress		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: TRUE : MCU Configuration to be used is fixed i.e. Mcu_ConfigRoot[0] FALSE: The user has to choose the MCU configuration to be used and pass the same as a parameter to Mcu_Init Default Values: FALSE		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter indicates whether the Mcu configuration to be used is decided at link time or changeable by user at runtime. #define MCU_PB_FIXEDADDR (STD_OFF/ON)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.8 Select Core for System Modes

Table 17 Select Core for System Modes

Syntax:	McuSystemModeCpuCore		
Source:	This parameter is IFX		
File:	Mcu_Cfg.h		
Range:	Values: CPU_CORE_0 – System Mode triggered by core 0 will be only considered CPU_CORE_1 – System Mode triggered by core 1 will be only considered CPU_CORE_2 – System Mode triggered by core 2 will be only considered UNANIMOUS_ALL_CORES – System mode has to triggered by all Cores Default Values: CPU_CORE_0		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter will define which core can trigger system modes (sleep / standby). This is not applicable for AURIX EP platforms.		
SFRs Modified	SCU_PMSWCR1		
Scope:	Driver.		
Dependency:	None		

4.2.9 Select Core for Idle Mode

Table 18 Select Core for System Modes

Syntax:	McuIdleModeCpuCore		
Source:	This parameter is IFX		
File:	Mcu_Cfg.h		
Range:	Values: CPU_CORE_0 – Core 0 will put all CPU's to IDLE mode CPU_CORE_1 – Core 1 will put all CPU's to IDLE mode CPU_CORE_2 – Core 2 will put all CPU's to IDLE mode INDIVIDUAL_CORES – Idle request by CoreX will put only CoreX in Idle mode. Default Values: INDIVIDUAL_CORES		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter will define which core can trigger idle mode. This is not applicable for AURIX EP platforms.		
SFRs Modified	SCU_PMSWCR1		
Scope:	Driver.		
Dependency:	None		

4.2.10 Initialize clock enable/disable

Table 19 Switch to enable/disable clock handling

Syntax:	McuInitClock		
Source:	This parameter is Autosar specific		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: Mcu is responsible for clock initializations. FALSE: Mcu is NOT responsible for clock initializations. Default Values: TRUE and is not editable.		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	If this parameter is set to FALSE, the clock initialization has to be disabled from the MCU driver. This concept applies when there are some write once clock registers and a bootloader is present. If this parameter is set to TRUE, the MCU driver is responsible of the clock initialization. Since clock initializations are mandatory for system to work, this parameter is permanently made stuck to TRUE and cannot be edited by the user. #define MCU_INIT_CLOCK_API (STD_OFF/ON)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.11 PLL handling enable/disable

Table 20 Switch to enable/disable PLL handling

Syntax:	McuNoPll		
Source:	This parameter is Autosar specific		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: Mcu does not have to intervene in PLL related setup. FALSE: Mcu is responsible to get the PLLs up and running. Default Values: FALSE and is not editable.		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter will be set True, if the H/W does not have a PLL or the PLL circuitry is enabled after the power on without S/W intervention. In this case Mcu_DistributePllClock has to be disabled and Mcu_GetPllStatus has to return MCU_PLL_STATUS_UNDEFINED. Otherwise this parameters has to be set False #define MCU_NO_PLL (STD_OFF/ON) Since PLL is present, this parameter is permanently made stuck to FALSE and cannot be edited by the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.12 GetRamState API Enable/Disable

Table 21 Enabling/Disabling GetRamState API

Syntax:	McuGetRamStateApi		
Source:	This parameter is Autosar specific		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: The function Mcu_GetRamState() is enabled. FALSE: The function Mcu_GetRamState() is disabled. Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the API Mcu_GetRamState. #define MCU_GET_RAM_STATE_API (STD_OFF/ON) The MCU driver returns the state of CPUx RAM before the last warm reset.		
SFRs Modified	None		
Scope:	Driver.		
Dependency:	None		

4.2.13 FM PLL Enable/Disable

Table 22 Enabling/Disabling FM PLL feature

Syntax:	McuFmPllEnable		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: Enable frequency modulation of PLL. FALSE: Disable frequency modulation of PLL. Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the frequency modulation of PLL. #define MCU_FMPLL_ENABLE (STD_OFF/ON)		
SFRs Modified	SCU_PLLCON0		
Scope:	Driver.		
Dependency:	None		

4.2.14 Mcu_ClearColdResetState API Enable/Disable

Table 23 Enabling/Disabling Mcu_ClearColdResetState API

Syntax:	McuClearColdResetApi		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: The function Mcu_ClearColdResetState () is enabled. FALSE: The function Mcu_ClearColdResetState () is disabled. Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the API Mcu_ClearColdResetState. #define MCU_CLR_COLD_RESET_STAT_API (STD_OFF/ON) The MCU driver clears the content of SCU_RSTSTAT register. Mcu driver clears cold reset bits of SCU_RSTSTAT register.		
SFRs Modified	SCU_RSTCON2		
Scope:	Driver.		
Dependency:	None		

4.2.15 Mcu_DeInit API Enable/Disable

Table 24 Enabling/Disabling Mcu_DeInit API

Syntax:	McuDeInitApi
Source:	This is IFX specific parameter

File:	Mcu_Cfg.h		
Range:	Values: TRUE: The function Mcu_DeInit () is enabled. FALSE: The function Mcu_DeInit () is disabled. Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the API Mcu_DeInit. #define MCU_DEINIT_API (STD_OFF/ON) This configuration parameter will enable availability for Mcu_DeInit API. Mcu_DeInit Api will de-initialize Mcu driver. This configuration parameter will also control de-initialization function for Gtm, Crc and Dma. For Mcu driver Mcu_DeInit will de-initialize only SFRs initialized by Mcu_Init API and it will not de-initialize PLL registers.		
SFRs Modified	SCU_PMSWCR1, SCU_RSTCON, SCU_ARSTDIS, PMSWCR0. If CRC is configured then FCE_KRST0, FCE_KRST1, FCE_CLC. If DMA is configured then DMA_CLC. GTM SFRs are mentioned in Gtm_DeInit API in GTM user manual.		
Scope:	Driver.		
Dependency:	None		

4.2.16 SFR Resetting at Initialization Enable/Disable

Table 25 Enabling/Disabling SFR resetting at Initialization

Syntax:	McuResetSfrAtInit		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: SFR Resetting at initialization will be enabled. FALSE: SFR Resetting at initialization will be disabled. Default Values: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the resetting of SFR at initialization for MCU and GTM module.		
SFRs Modified	Modified GTM SFRs are mentioned in Gtm_Init api in GTM user manual.		
Scope:	Driver.		
Dependency:	None		

4.2.17 Protected mode Enable/Disable for Initialization APIs

Table 26 Enabling/Disabling Protected mode register access in all Init APIs

Syntax:	McuUserModeInitApiEnable

Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: FALSE: Default Value: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable / disable protected register access in all the Init APIs		
SFRs Modified	None		
Scope:	Driver.		
Dependency:	None		

4.2.18 Protected mode Enable/Disable for De-initialization APIs

Table 27 Enabling/Disabling Protected mode register access in all De-init APIs

Syntax:	McuUserModeDeInitApiEnable		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: FALSE: Default Value: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable / disable protected register access in all the DeInit API		
SFRs Modified	None		
Scope:	Driver.		
Dependency:	None		

4.2.19 Protected mode Enable/Disable during Runtime APIs

Table 28 Enabling/Disabling Protected mode register access in all Runtime APIs

Syntax:	McuUserModeRuntimeApiEnable		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: FALSE: Default Value: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--

Description:	Pre-processor switch to enable / disable protected register access in all the Runtime APIs other than Init/Deinit APIs
SFRs Modified	None
Scope:	Driver.
Dependency:	None

4.2.20 Protected mode Enable/Disable

Table 29 Enabling/disabling the Normal/Protected mode

Syntax:	McuRunningInUser0Mode		
Source:	This is IFX specific parameter		
File:	Mcu_Cfg.h		
Range:	Values: TRUE: FALSE: Default Value: FALSE		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the Normal/Protected mode		
SFRs Modified	None		
Scope:	Driver.		
Dependency:	None		

4.3 Container: Mcu Module Configuration

This is a multiple-configuration container. However, when McuPBFixedAddress is set to true, only one configuration is possible.

4.3.1 MCU Clock Source Failure Notification Configuration

This parameter relates to the clock source failure notification configuration.

Table 30 MCU Clock Source Failure Notification Configuration

Name:	McuClockSrcFailureNotification		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	Values: ENABLED – Enable function to report DEM DISABLED – Disable function to report DEM Default value: DISABLED		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	-
Description:	This Enables/Disables clock failure notification function. The PLL may become unlocked, caused by a break of the crystal or the external clock line. In such a case, an SMU alarm is generated if the corresponding SMU alarm is		

	enabled. On the SMU alarm, the user can invoke this notification function which in turn reports a DEM.
SFRs Modified	None
Scope:	Driver
Dependency:	None

If Enabled, the DEM configuration has to done in the container McuDemEventParameterRefs

4.3.2 Mcu Number of Modes Configuration

This parameter relates to the Number of modes configured.

Table 31 McuNumberOfMcuModes

Name:	McuNumberOfMcuModes		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	Values: 3 and not editable		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	-	
Description:	This parameter represents the number of Modes available for the MCU. #define MCU_MAX_NO_MODES 3 *Note: The parameter McuNumberOfMcuModes is disabled always since it is calculated manually in the configuration. (**Refer 4.7.1**)*		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.3 McuRamSectors Configuration

This parameter relates to the number of RamSectors configured.

Table 32 McuRamSectors

Name:	McuRamSectors		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	Values: <count> Default value: 0		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter represents the number of RAM sectors available for the MCU. #define MCU_RAM_SECTORS (<count>) *Note: The parameter McuRamSectors is disabled always since it is calculated manually in the configuration. (**Refer 4.7.3**)*		
SFRs Modified	None		
Scope:	Driver		

Dependency:	None

4.3.4 MCU SW Reset Configuration

This parameter relates to the MCU software reset configuration. This applies to the function Mcu_PerformReset (software reset), which performs a microcontroller reset.

Table 33 MCU SW Reset configuration

Name:	McuResetSetting		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_PBCfg.c		
Range:	Values: 0 – No RESET 1 - SYSTEM RESET 2 – APPLICATION RESET Default value: 0 – NO RESET if McuPerformResetApi is FALSE 1 – SYSTEM RESET if McuPerformResetApi is TRUE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	MCU software reset configuration is done with this configuration. This configuration will be used by the function Mcu_PerformReset, which performs a microcontroller reset. This value will be used to update the SW bit in RSTCON register. The generated value is stored in the (8,9) bits of ResetCfg which is an element in Mcu_ConfigType structure.		
SFRs Modified	SCU_RSTCON		
Scope:	Driver		
Dependency:	McuPerformResetApi		

4.3.5 Container: MCU Trigger Reset Configuration

This McuTriggerReset container relates to the SCU specific reset configuration other than software reset. It provides configuration parameters for different types of reset trigger available and type of reset each trigger can perform. The SW reset trigger applies to the function Mcu_PerformReset (software reset) and this configuration is done by **MCU SW Reset Configuration**. The multiplicity of this container is 1:1.

Table 34 MCU Trigger Reset configuration

Name:	<Type>ResetTrigger		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: 0 – NO RESET GENERATED 1 - SYSTEM RESET 2 – APPLICATION RESET Default value: 0 – NO RESET GENERATED		
Configuration class	Pre-compile	--	--
	Link Time	--	--

	Post build	X	VariantPB
Description:	The following types <Type> of reset triggers are: SMU, ESR0, ESR1, STM0, STM1 (if CPU1 exists) and STM2 (if CPU2 exists). These configuration parameters will be generated to configure the SCU_RSTCON register to decide which are the available reset triggers for the system and kind of reset they can generate. The generated value is stored in the lower 16 bits of ResetCfg which is an element in Mcu_ConfigType structure.		
SFRs Modified	SCU_RSTCON		
Scope:	Driver		
Dependency:	None		

4.3.6 MCU Stm Configuration

This parameter relates to the system timer configuration. An application reset can either have no effect or resets the respective STM timers.

Table 35 MCU Stm configuration

Name:	McuStm<Num>ResetDisable		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: FALSE – Resets the STM TRUE – Does not reset the STM Default value: FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This configuration parameter decides whether to reset the STM timer on a microcontroller application reset. The <Num> represents the STM timer per core and the values are 0, 1 (if CPU1 exists) and 2 (if CPU2 exists). The generated value is stored in the upper 16 bits of ResetCfg which is an element in Mcu_ConfigType structure.		
SFRs Modified	SCU_ARSTDIS		
Scope:	Driver		
Dependency:	None		

4.3.7 Container: MCU DMA Configuration

This container DmaConfiguration provides DMA channel allocation to each driver and to ensure no resource clash occurs. If this container has children, then only MCU will initialize the DMA clock (DMA_CLC) and DMA Error Interrupt Set register (DMA_ERRINTR).

4.3.7.1 DMA Channel

DmaChannel is a sub container which has a list of DMA channels (Refer **Table 6** for number of DMA channels per derivative). Each channel can be allocated for a specific driver.

The Mcu module doesn't contain any source code changes/template file changes for the container (except the .xdm file). Validation for the channel allocation will be done in respective module drivers.

4.3.7.1.1 ChannelId

The DMA channel identifier.

Table 36 DMA Channel Id

Name:	ChannelId		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Values: 0 – x where x is maximum number of DMA channels available per derivative		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	DMA Channel number		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.7.1.2 ChannelUsed

This is an indicator as to who owns this DMA channel.

Table 37 DMA Channel Used

Name:	ChannelUsed		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Values: USE_FOR_NONE USE_FOR_SPI_DRIVER USE_FOR_ADC_DRIVER USE_FOR_DMA_CDD Default value: USE_FOR_NONE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	Each channel can be allocated to a specific driver mentioned above. The respective drivers have to use this as a one point global resource allocator.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.8 Container: GtmConfiguration

If this container has children, then only MCU will initialize GTM. Refer **[5]** for GTM related configuration

4.3.9 Container: CcuConfiguration

CCU6 channel allocation to each driver is done in MCU driver configuration. A CCU6 Container (CcuConfiguration) configuration is added. This CcuConfiguration container contains list of CC channels under individual CCU6x modules. Each module can be allocated for a specific driver.

Allocation of CC channels to modules happen in the kernel/module level i.e. if CC62 of CCU60 is to be used for ICU then the whole CCU60 has to be allocated to ICU driver, and it becomes ICU driver's responsibility to perform the T12 and CC6x initializations of that CCU6x module/kernel. The other channels of the CCU6x module should not be used by any other driver.

The Mcu module doesn't contain any source code changes/template file changes for the Container (except the .xdm file). Validation for the Channel allocation will be done in respective modules.

4.3.9.1 Ccu6ModuleUsage

Table 38 CCU6 Module Usage

Name:	Ccu6ModuleUsage		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	UNUSED USED_BY_ICU_DRIVER USED_BY_ADC_DRIVER Default : UNUSED		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is used to avoid Resource conflict in usage of CC channels. CCU6x kernels are individually allocated to different drivers and the CC channels within that CCU6x kernel will altogether belong to that specific driver. Also, the responsibility to initialize the resources of this kernel will be handled by that chosen driver alone.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.9.2 T12ClkSelection

Table 39 T12 Clock Selection

Name:	T12ClkSelection		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	0 to 7 Default : 0		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is used to configure the clock divider for T12 timer.		
SFRs Modified	CCU6_TCTR0 (Refer UM of ICU driver for more details)		
Scope:	Driver		

Dependency:	None

4.3.9.3 T13ClkSelection

Table 40 T13 Clock Selection

Name:	T13ClkSelection		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	0 to 7 Default : 0		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is used to configure the clock divider for T13 timer.		
SFRs Modified	CCU6_TCTR0 (Refer UM of ICU/ADC driver for more details)		
Scope:	Driver		
Dependency:	None		

4.3.9.4 T12PrescalerEnabled

Table 41 T12 Prescaler Enabled

Name:	T12PrescalerEnabled		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	TRUE, FALSE Default : FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is used to configure if divide by 256 pre-scalar is required or not.		
SFRs Modified	CCU6_TCTR0 (Refer UM of ICU driver for more details)		
Scope:	Driver		
Dependency:	None		

4.3.9.5 T13PrescalerEnabled

Table 42 T13 Prescaler Enabled

Name:	T13PrescalerEnabled		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	TRUE, FALSE Default : FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--

	Post build	X	VariantPB
Description:	This parameter is used to configure if divide by 256 pre-scalar is required or not.		
SFRs Modified	CCU6_TCTR0 (Refer UM of ICU/ADC driver for more details)		
Scope:	Driver		
Dependency:	None		

4.3.9.6 CCChannelInputSelection

Table 43 CC Channel Input Selection

Name:	CCChannelInputSelection		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Depends on the micro-controller Default : Value according to default value of the register		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is used to choose the input for the CC channel.		
SFRs Modified	CCU6_PISEL0 (Refer UM of ICU driver for more details)		
Scope:	Driver		
Dependency:	None		

4.3.10 AscLinConfiguration

4.3.10.1 AscLin<x>

ASCLIN allocation to each driver is done in MCU driver configuration; hence ASCLIN Container (AscLinConfiguration) configuration is added. This AscLinConfiguration container contains list of ASCLIN's. Each instance can be allocated for a specific driver.

The Mcu module doesn't contain any source code changes/template file changes for the Container (except the .xdm file). Validation for the Channel allocation will be done in respective module drivers.

There are four AscLin's in AURIX HE family: AscLin0, AscLin1, AscLin2 and AscLin3.

There are two AscLin's in AURIX EP family: AscLin0 and AscLin1.

Table 44 AscLin<x>

Name:	AscLin<x> where x is number of ASCLIN's per derivative		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Values: USE_FOR_NONE USE_FOR_UART_DRIVER USE_FOR_LIN_DRIVER USE_FOR_STDLIN_DRIVER Default value: USE_FOR_NONE		
Configuration	Pre-compile	--	--

class	Link Time	--	--
	Post build	X	VariantPB
Description:	Each module can be allocated to a specific peripheral device mentioned above. The respective peripheral devices have to use this as a one point global resource allocator.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.11 EruConfiguration

4.3.11.1 ERU<x>

ERU allocation to each driver is done in MCU driver configuration; hence ERU Container (EruConfiguration) configuration is added. This EruConfiguration container contains list of ERU's. Each instance can be allocated for a specific driver.

The Mcu module doesn't contain any source code changes/template file changes for the Container (except the .xdm file). Validation for the ERU allocation will be done in respective modules.

There are 8 ERU's; ERU<x> where x is 0 to 7.

Table 45 EruChannelUsage

Name:	EruChannelUsage		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Values: UNUSED USE_BY_ADC_DRIVER USED_BY_ICU_DRIVER USED_BY_OTHER_DRIVER Default value: UNUSED		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	Each module mentioned above can be allocated to a specific ERU unit. The respective modules have to use this as a one point global resource allocator.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

Table 46 EruInputLine

Name:	EruInputLine		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Refer respective target derivative's UM **[3]**, **[6]** or **[7]** for the respective ERU input lines		
Configuration	Pre-compile	--	--

class	Link Time	--	--
	Post build	X	VariantPB
Description:	Selection of input line for the respective ERU unit.		
SFRs Modified	SCU_EICR<x> where x = 0,1,2,3 (Refer UM of the driver assigned to this ERU unit)		
Scope:	Driver		
Dependency:	None		

Table 47 EruOutputUnit

Name:	EruOutputUnit		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Values: ERU_OUTPUT_GATING_UNIT0 ERU_OUTPUT_GATING_UNIT1 ERU_OUTPUT_GATING_UNIT2 ERU_OUTPUT_GATING_UNIT3 ERU_OUTPUT_GATING_UNIT4 ERU_OUTPUT_GATING_UNIT5 ERU_OUTPUT_GATING_UNIT6 ERU_OUTPUT_GATING_UNIT7 Default value: ERU_OUTPUT_GATING_UNIT0		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	Selection of output unit for the respective ERU unit.		
SFRs Modified	SCU_IGCR<x> where x = 0,1,2,3 (Refer UM of the driver assigned to this ERU unit)		
Scope:	Driver		
Dependency:	None		

4.3.12 Container: MCU RAM Sector Setting Configuration

This container contains the configuration parameters for the RAM sector setting.

Please ensure unique short-name for this container in every instance of Mcu configuration.

4.3.12.1 RAM Section Base Address

Table 48 RAM section Base Address

Name:	McuRamSectionBaseAddress		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_PBCfg.c		
Range:	Values: Refer **Table 2** for RAM memory ranges Default: 70000000		
Configuration class	Pre-compile	--	--
	Link Time	--	--

	Post build	X	VariantPB
Description:	This parameter represents the MCU RAM Section Base Address. This will be a member of Mcu_RamCfgType structure.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.12.2 RAM Section Default Data Value

Table 49 RAM section default data value

Name:	McuRamDefaultValue		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_PBCfg.c		
Range:	Values: 0x00...0xFF Default value: 0		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is the preset value used to fill the configured RAM section. This will be a member of Mcu_RamCfgType structure.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.12.3 RAM Section Size

Table 50 RAM section size

Name:	McuRamSectionSize		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu.h		
Range:	McuRamSectionBaseAddress+ McuRamSectionSize should not exceed boundary for RAM Section. Refer **Table 2** for the RAM ranges. Default value: 8192		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter represents the MCU RAM Section size in bytes. This will be a member of Mcu_RamCfgType structure.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuRamSectionBaseAddress, McuRamSectionSize		

4.3.13 Container: MCU Clock Setting Configuration

Please ensure unique short-name for this container in every instance of Mcu configuration.

4.3.13.1 McuClockSettingId for clock identification

Table 51 McuClockSettingId clock identification

Name:	McuClockSettingId		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	0 – 255		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	The Id of this McuClockSettingConfig to be used as argument for the API call "Mcu_InitClock". The value's given to different clock settings, should start from 0 and must be consecutive. (0, 1, 2, 3…244, 255). This is an open point in Bugzilla- 48322		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

This container contains the configuration parameters for the determining the System Clock.

4.3.13.2 Container: McuClockReferencePoint

This container defines parameters responsible to generate system clock and clocks for sub-components of Clock Control Unit.

In case default values are not used, it is recommended to use [3] to generate the frequencies and divider values for all the clocks. Refer Table 3 and Table 4 to configure the divider values for all the clocks.

4.3.13.3 McuPllPDivider

Table 52 PLL parameter P-Divider configuration

Name:	McuPllPDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0...15 [Fosc(min) / Fref(max) .. Fosc(max) / Fref(min)] Default Value: 1 for AURIX HE 0 for AURIX EP		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	Frequency divider of main oscillator (4 bits). McuPllPDivider is a member under Mcu_ClockCfgType and is the value written into the SFR. Clock equations need the addition of 1 to this parameter. Refer **2.2.2.3** for clock calculation. It should be selected such that, Fref is in valid range.		

SFRs Modified	SCU_PLLCON0
Scope:	Driver
Dependency:	None

4.3.13.4 McuPllNDivider

Table 53 PLL parameter N-Divider configuration

Name:	McuPllNDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0...127 [Fvco(min) / Fref(max) .. Fvco(max) / Fref(min)] Default Value: 47 for AURIX HE 29 for AURIX EP		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	7 bit feedback divider value used for generation of system clock. McuPllNDivider is a member under Mcu_ClockCfgType and is the value written into the SFR. Clock equations need the addition of 1 to this parameter. Refer **2.2.2.3** for clock calculation. It should be selected such that Fvco is in valid range.		
SFRs Modified	SCU_PLLCON0		
Scope:	Driver		
Dependency:	McuPllBypass		

4.3.13.5 McuPllK1Divider

Table 54 PLL parameter K1-Divider configuration

Name:	McuPllK1Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..127 Default Value: 1		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	7 bit output divider. Even values are preferred to get 50% duty cycle. McuPllK1Divider is a member under Mcu_ClockCfgType and is the value written into the SFR. Clock equations need the addition of 1 to this parameter. Refer **2.2.2.3** for clock calculation. *Note: Changing the system operation frequency by changing the value of the K1-Divider has a direct coupling to the power consumption of the device. Therefore this has to be done carefully.* This divider is used only when PLL is configured in “Pre-scalar Mode”		
SFRs Modified	SCU_PLLCON1		

Scope:	Driver
Dependency:	McuPllBypass

4.3.13.6 McuPllK2Divider

Table 55 PLL parameter K2-Divider configuration

Name:	McuPllK2Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0...127 [Fvco(min) / Fpll(max) .. Fvco(max) / Fpll(min)] Default Value: 2 for AURIX HE 4 for AURIX EP		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	7 bit output divider. Even values are preferred to get 50% duty cycle. McuPllK2Divider is a member under Mcu_ClockCfgType and is the value written into the SFR. Clock equations need the addition of 1 to this parameter. Refer **2.2.2.3** for clock calculation. *Note: Changing the system operation frequency by changing the value of the K2-Divider has a direct coupling to the power consumption of the device. Therefore this has to be done carefully.* It should be selected such that Fpll should be in valid range.		
SFRs Modified	SCU_PLLCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.7 McuK2DivSteps

Table 56 PLL parameter K2-Divider Steps configuration

Name:	McuK2DivSteps		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..6, when McuClockSettingMode is NORMAL MODE Default Value: 4		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	4 bit step value. This parameter gives the intermediate step value to achieve the final target frequency. K2steps is a member under Mcu_ClockCfgType. Note: If McuK2DivSteps is configured with a higher value for smaller deltas in the frequency then there is a possibility of redundant jumps to the same frequency as the result of lesser granularity (for SCU_PLLCON1.B. K2DIV). User has a choice to reduce the number of steps to avoid redundant jumps. However,		

	redundant jumps do not cause any harm in achieving the final frequency.
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.3.13.8 McuK2DivRampToPllConfDelay

Table 57 Mcu K2 Divider Ramp To PLL Configurable Delay

Name:	McuK2DivRampToPllConfDelay		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 5us ..100us Default Value: 10us		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter denotes the delay configured for K2 ramp up/down to PLL frequency. The delay generated depends on the intermediate frequencies of K2 divider steps. This parameter is used to generate delay ticks which are used in Mcu_InitClock and Mcu_DistributePLL when ramping up / ramping down PLL to the desired frequency in normal mode. K2RampToPllDelayTicks is a member under Mcu_ClockCfgType. ***Note: The configured delay value is used to generate delay loops based out of software. The delay values may not be accurate due to compiler optimizations / CPU performance and could give a delay time greater than the configured value. It is recommended to fine tune this value based on the same to get accurate delays.***		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.13.9 McuK2DivRampToBackUpConfDelay

Table 58 Mcu K2 Divider Ramp To Backup Configurable Delay

Name:	McuK2DivRampToBackUpConfDelay		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 5us ..100us Default Value: 10us		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	32 bit value. This parameter denotes the delay configured for K2 ramp To backup clock frequency. The delay generated depends on the intermediate frequencies of K2 divider steps.		

	This parameter is used to generate delay ticks which are used in Mcu_RampToBackUpClockFreq when ramping PLL close to the backup clock. K2RampToBackUpDelayTicks is a member under Mcu_ClockCfgType. ***Note: The configured delay value is used to generate delay loops based out of software. The delay values may not be accurate due to compiler optimizations / CPU performance and could give a delay time greater than the configured value. It is recommended to fine tune this value based on the same to get accurate delays.***
SFRs Modified	None
Scope:	Driver
Dependency:	McuMode

4.3.13.10 McuPllK3Divider

Table 59 PLL parameter K3-Divider configuration

Name:	McuPllK3Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0...127 [Fvco(min) / Fpll(max) .. Fvco(max) / Fpll(min)] Default Value: 5		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB
Description:	7 bit output divider. Even values are preferred to get 50% duty cycle. McuPllK3Divider is a member under Mcu_ClockCfgType and is the value written into the SFR bit field. Clock equations need the addition of 1 to this parameter. Refer **2.2.2.3** for clock calculation.		
SFRs Modified	SCU_PLLCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.11 McuFMPllModAmp

Table 60 FM PLL Amplitude configuration

Name:	McuFMPllModAmp		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0 - 5 Default Value: 1.25		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	It is the percentage value for Modulation Amplitude for PLL Frequency modulation. This value is equated as (64 * McuFMPllModAmp/100 * McuMainOscillatorFrequency/(McuPllPDivider + 1) * (McuPllNDivider + 1)/3.6); units: McuFMPllModAmp [%] , McuMainOscillatorFrequency [MHz] and corresponds to		

	MODCFG [9:0] bits of SCU_PLLCON2. Refer to the Application Note [9] for more details.
SFRs Modified	SCU_PLLCON2
Scope:	Driver
Dependency:	McuMainOscillatorFrequency, McuPllPDivider, McuPllNDivider

4.3.13.12 McuClockSettingMode

Table 61 McuClockSettingMode configuration

Name:	McuClockSettingMode		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 1 : NORMAL_MODE 0 : PRESCALER_MODE Default value: NORMAL_MODE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	K2DIV, K3DIV, NDIV and PDIV values do not have any effect if Pre-scalar Mode is selected. PllMode is a member under Mcu_ClockCfgType		
SFRs Modified	SCU_PLLCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.13 McuCPU0Divider

Table 62 CPU0 Divider value

Name:	McuCPU0Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..63 Default Value: 32 for AURIX HE 0 for AURIX EP		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	CPU0 divider value used for generation of CPU0 clock. Fcpu0 = Fsri * (64-McuCPU0Divider) /64.		
SFRs Modified	SCU_CCUCON6		
Scope:	Driver		
Dependency:	None		

4.3.13.14 McuCPU1Divider

Table 63 CPU1 Divider value

Name:	McuCPU1Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..63 Default Value: 32		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB
Description:	CPU1 divider value used for generation of CPU1 clock. Fcpu1 = Fsri * (64-McuCPU1Divider) /64. This is not applicable for AURIX EP platforms.		
SFRs Modified	SCU_CCUCON7		
Scope:	Driver		
Dependency:	None		

4.3.13.15 McuCPU2Divider

Table 64 CPU2 Divider value

Name:	McuCPU2Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..63 Default Value: 32		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	VariantPB
Description:	CPU2 divider value used for generation of CPU2 clock. Fcpu2 = Fsri * (64-McuCPU2Divider) /64. This parameter will not be available for TC26x variant. This is not applicable for AURIX EP platforms.		
SFRs Modified	SCU_CCUCON8		
Scope:	Driver		
Dependency:	None		

4.3.13.16 McuSRIDivider

Table 65 SRI Divider value

Name:	McuSRIDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 1,2,3,4,5,6,8,10,12 and 15 Default Value: 1		
Configuration class	Pre-compile	--	--
	Link Time	--	--

	Post build	X	VariantPB
Description:	SRI divider value used for SRI domain i.e. CPU1, CPU2, DMA, SDMA etc. Fsri = (Fpll / McuSRIDivider).		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.17 McuSPBDivider

Table 66 SPB divider value

Name:	McuSPBDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	SPB divider value used for generation of system clock for peripheral/spb domain. Fspb = (Fpll / McuSPBDivider). Only even values can be configured.		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.18 McuFSIDivider

Table 67 FSI divider value

Name:	McuFSIDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	FSI divider value used for generation of FSI clock. 0 Ffsi is shut off 1 Ffsi = Fsri 2 Ffsi = Fsri / 2 for SRIDIV = 0001B or 0010B, else Ffsi = Fsri 3 Ffsi = Fsri / 3 for SRIDIV = 0001B or 0010B, else Ffsi = Fsri		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.19 McuFSI2Divider

Table 68 FSI2 Divider Value

Name:	McuFsi2Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2 and 3 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	0 Ffsi2 is shut off 1 Ffsi2 = Fsri 2 Ffsi2 = Fsri / 2 for SRIDIV = 0001B or 0010B, else Ffsi2 = Fsri 3 Ffsi2 = Fsri / 3 for SRIDIV = 0001B or 0010B, else Ffsi2 = Fsri		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.20 McuMAXDivider

Table 69 MAX divider value

Name:	McuMAXDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values:1,2,3,4,5,6,8,10,12 and 15 Default Value: 1		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	MAX divider value used for generation of MAX clock. Fmax = Fpll / (McuMAXDivider)		
SFRs Modified	SCU_CCUCON5		
Scope:	Driver		
Dependency:	None		

4.3.13.21 McuEBUDivider

Table 70 EBU divider value

Name:	McuEBUDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values:0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 0 (Not used)		
Configuration	Pre-compile	--	--

class	Link Time	--	--
	Post build	X	VariantPB
Description:	EBU divider value used for generation of EBU clock. Febu = Fplleray / (McuEBUDivider). This parameter will be available for TC29x only.		
SFRs Modified	SCU_CCUCON5		
Scope:	Driver		
Dependency:	None		

4.3.13.22 McuSTMDivider

Table 71 STM divider value

Name:	McuSTMDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	STM divider value used for generation of STM clock. Fstm = Fpll / (McuSTMDivider)		
SFRs Modified	SCU_CCUCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.23 McuGTMDivider

Table 72 GTM divider value

Name:	McuGTMDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	GTM divider value used for generation of GTM clock. FGtm = Fpll / (McuGTMDivider)		
SFRs Modified	SCU_CCUCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.24 McuBBBDivider

Table 73 BBB Divider value

Name:	McuBBBDivider

Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	BBB divider value used for generation of BBB clock. Fbbb= Fpll / (McuBBBDivider)		
SFRs Modified	SCU_CCUCON2		
Scope:	Driver		
Dependency:	None		

4.3.13.25 McuBAUD1Divider

Table 74 BAUD1 Divider Value

Name:	McuBAUD1Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	BAUD1 divider value used for generation of BAUD1 clock. FBaud1 = Fpll / (McuBAUD1Divider). This is not applicable for TC23x/TC22x/TC21x platforms.		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.26 McuBAUD2Divider

Table 75 Baud2 Divider Value

Name:	McuBAUD2Divider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	BAUD2 divider value used for generation of BAUD2 clock. FBaud2 = Fpll / (McuBAUD2Divider)		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.27 McuCANDivider

Table 76 CAN divider value

Name:	McuCANDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	CAN divider value used for generation of CAN clock. FCan = Fpll / (McuCANDivider)		
SFRs Modified	SCU_CCUCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.28 McuASCLINSDivider

Table 77 Asclins divider value

Name:	McuASCLINSDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 8 for AURIX HE 2 for AURIX EP		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	ASCLINS divider value used for generation of ASCLINS clock. FAsclins = Fpll / (McuASCLINSDivider)		
SFRs Modified	SCU_CCUCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.29 McuASCLINFDivider

Table 78 Asclinf divider value

Name:	McuASCLINFDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0,1,2,3,4,5,6,8,10,12 and 15 Default Value: 2		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB

Description:	ASCLINF divider value used for generation of ASCLINF clock. FAsclins = Fpll / (McuASCLINFDivider)
SFRs Modified	SCU_CCUCON1
Scope:	Driver
Dependency:	None

4.3.13.30 McuClockASCLINSFrequency

Table 79 Asclins clock frequency

Name:	McuClockASCLINSFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 20MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of <ASCLINS-Frequency>, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use the MCC XLS provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuAsclinsDivider		

4.3.13.31 McuClockASCLINFFrequency

Table 80 Asclinf clock frequency

Name:	McuClockASCLINFFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of <ASCLINF-Frequency>, can refer the McuClockReferencePoint container and fetch the value from this parameter.		

	The user will just need to enter the frequency here. **Caution**: It is recommended to use the MCC XLS provided as this parameter is not validated against safe values/ranges.
SFRs Modified	None
Scope:	Driver
Dependency:	McuAsclinfDivider

4.3.13.32 McuClockSelectADC

Table 81 ADC Clock Select

Name:	McuClockSelectADC		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Range: ADC_CLOCK_OFF : ADC Clock is off ADC_CLOCK_PLL2 : ADC clock derived from PLL2 ADC_CLOCK_ERAYPLL2 : ADC clock is derived from ERAY PLL2 ADC_CLOCK_BACKUP : ADC clock is derived from backup clock Default Value: ADC_CLOCK_PLL2 Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter defines the input clock for ADC peripheral. This is not applicable for TC23x/TC22x/TC21x platforms.		
SFRs Modified	SCU_CCUCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.33 McuClockCANFrequency

Table 82 CAN clock frequency

Name:	McuClockCANFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of <CAN-Frequency>, can refer the McuClockReferencePoint container and fetch the value		

	from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use the MCC XLS provided as this parameter is not validated against safe values/ranges.
SFRs Modified	None
Scope:	Driver
Dependency:	McuCanDivider

4.3.13.34 McuClockSPBFrequency

Table 83 SPB clock frequency

Name:	McuClockSPBFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of <SPB-Frequency>, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use [3] provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuSPBDivider		

4.3.13.35 McuClockFSIFrequency

Table 84 FSI clock frequency

Name:	McuClockFSIFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only.		

	Any peripheral that needs to do a computation which is a function of <FSI-Frequency>, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use [3] provided as this parameter is not validated against safe values/ranges.
SFRs Modified	None
Scope:	Driver
Dependency:	McuFSIDivider

4.3.13.36 McuClockFSI2Frequency

Table 85 FSI2 clock frequency

Name:	McuClockFsi2Frequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of <FSI2-Frequency>, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use [3] provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuFsi2Divider		

4.3.13.37 McuClockCPU0Frequency

Table 86 CPU0 clock frequency

Name:	McuClockCPU0Frequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 120MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration	Pre-compile	--	--

class	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of CPU0-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuCPU0Divider		

4.3.13.38 McuClockCPU1Frequency

Table 87 CPU1 clock frequency

Name:	McuClockCPU1Frequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter defines the operating frequency for CPU1. This is not applicable for AURIX EP platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuCpu1Divider		

4.3.13.39 McuClockCPU2Frequency

Table 88 CPU2 clock frequency

Name:	McuClockCPU2Frequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter defines the operating frequency for CPU2. This parameter will not be available for TC26x. This is not applicable for AURIX EP platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuCpu2Divider		

4.3.13.40 McuClockMAXFrequency

Table 89 MAX clock frequency

Name:	McuClockMAXFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 160MHz for AURIX HE 120MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of MAX-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use [3] provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuMaxDivider		

4.3.13.41 McuClockEBUFrequency

Table 90 EBU clock frequency

Name:	McuClockEBUFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 0 (Not used) Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of MAX-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuMaxDivider		

4.3.13.42 McuClockGTMFrequency

Table 91 GTM clock frequency

Name:	McuClockGTMFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of GTM-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuGTMDivider		

4.3.13.43 McuClockSRIFrequency

Table 92 SRI clock frequency

Name:	McuClockSRIFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 160MHz for AURIX HE 120MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of SRI-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. **Caution**: It is recommended to use [3] provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuSRIDivider		

4.3.13.44 McuClockSTMFrequency

Table 93 STM clock frequency

Name:	McuClockSTMFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of STM-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuSTMDivider		

4.3.13.45 McuClockReferencePointFrequency

Table 94 PLL Frequency

Name:	McuClockReferencePointFrequency (also known as Fpll or Fsource)		
Source:	This parameter is defined by AUTOSAR		
File:	No generation		
Range:	Default Value: 160MHz for AURIX HE 120MHz for AURIX EP Refer **Table 3** for the range in NORMAL_MODE. For PRESCALER_MODE, range will be 0 to 40MHz.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of PLL-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. *Note:* The user will just need to enter the frequency here. **Caution**: It is recommended to use **[3]** provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.13.46 McuClockReferencePoint2Frequency

Table 95 PLL2 Frequency

Name:	McuClockReferencePoint2Frequency (also known as Fpll2)		
Source:	This parameter is defined by IFX		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of PLL2-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. *Note:* The user will just need to enter the frequency here. **Caution**: It is recommended to use the MCC XLS provided as this parameter is not validated against safe values/ranges.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.13.47 McuClockBBBFrequency

Table 96 BBB clock frequency

Name:	McuClockBBBFrequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of BBB-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. Note – This clock is available only in ED devices.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuBbbDivider		

4.3.13.48 McuClockBAUD1Frequency

Table 97 BAUD1 clock frequency

Name:	McuClockBAUD1Frequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of BAUD1-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here. This is not applicable for TC23x/TC22x/TC21x platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuBaud1Divider		

4.3.13.49 McuClockBAUD2Frequency

Table 98 BAUD2 clock frequency

Name:	McuClockBAUD2Frequency		
Source:	This is IFX specific parameter.		
File:	No generation		
Range:	Default Value: 80MHz for AURIX HE 60MHz for AURIX EP Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. Any peripheral that needs to do a computation which is a function of BAUD2-Frequency, can refer the McuClockReferencePoint container and fetch the value from this parameter. The user will just need to enter the frequency here.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuBaud2Divider		

4.3.13.50 McuErayPllPDivider

Table 99 ERAY Pll P Divider

Name:	McuErayPllPDivider		
Source:	This is IFX specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..15 Default Value: 0		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	Frequency divider of main oscillator (4 bits). McuPllPDivider is a member under Mcu_ClockCfgType and is the value written into the SFR. Clock equations need the addition of 1 to this parameter. It should be selected such that, Fref (Fosc/PDIV) for Eray PLL is in valid range. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	SCU_PLLERAYCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.51 McuErayPllNDivider

Table 100 ERAY Pll N divider

Name:	McuErayPllNDivider		
Source:	This is vendor specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..31 Default Value : 19		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	It is the 6 bit feedback divider value used for generation of Eray PLL clock. It has to select Fvco for ERAY Pll is in valid range. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	SCU_PLLERAYCON0		
Scope:	Driver		
Dependency:	None		

4.3.13.52 McuErayPllK2Divider

Table 101 ERAY Pll K2 divider

Name:	McuPllK2Divider
Source:	This is vendor specific parameter.
File:	Mcu_PBCfg.c

Range:	Values: 0...127 [Fvco(min) / Feraypll(max) .. Fvco(max) / Feraypll(min)] Default Value: 4		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	7 bit output divider. Even values are preferred to get 50% duty cycle. McuErayPllK2Divider is a member under Mcu_ClockCfgType and is the value written into the SFR. Clock equations need the addition of 1 to this parameter. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	SCU_PLLERAYCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.53 McuErayPllK3Divider

Table 102 McuErayPllK3Divider

Name:	McuErayPllK3Divider		
Source:	This is vendor specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0..15 Default Value : 4		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	It is the 4 bit feedback divider value used for generation of Eray PLL clock This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	SCU_PLLERAYCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.54 McuClockErayPllFrequency

Table 103 ERAY Pll Frequency

Name:	McuClockErayPllFrequency (Frequency output of the ERAY PLL)		
Source:	This parameter is defined by IFX.		
File:	No generation		
Range:	Default: 80 MHz Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. It is final frequency ouput of the ERAY Pll. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	None		

Scope:	Driver
Dependency:	None

4.3.13.55 McuClockErayPll2Frequency

Table 104 ERAY Pll2 Frequency

Name:	McuClockErayPll2Frequency (Frequency output of the ERAY PLL 2)		
Source:	This parameter is defined by IFX.		
File:	No generation		
Range:	Default: 80 MHz Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. It is final frequency output of the ERAY PLL2. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.13.56 McuErayDivider

Table 105 ERAY Clock Divider

Name:	McuErayClockDivider		
Source:	This is vendor specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: 0 – 15 Default : 1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	ERAY module clock configuration (Runtime Mode Control setting) Eray Divider is used to generate the Feray = Feraypll / McuErayDivider. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	SCU_CCUCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.57 McuClockErayFrequency

Table 106 ERAY Frequency

Name:	McuClockErayFrequency (ERAY frequency)

Source:	This parameter is defined by IFX.		
File:	Mcu_PBCfg.c		
Range:	Default : 80 MHz Refer **Table 3** for the range.		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. It is final frequency given to the Eray module. This is not applicable for TC22x/TC21x platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.13.58 McuEthDivider

Table 107 Vendor specific configuration parameter: McuEthDivider

Name:	McuEthDivider		
Source:	This is vendor specific parameter.		
File:	Mcu_PBCfg.c		
Range:	Values: Values: 0,1,2,3,4,5,6,8,10,12,15 Default : 1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Eth module clock configuration. McuEthDivider Divider is used to generate the Feth = Fraypll / (McuEthDivider * 4). This is not applicable for TC23x/TC22x/TC21x platforms.		
SFRs Modified	SCU_CCUCON1		
Scope:	Driver		
Dependency:	None		

4.3.13.59 McuClockEthFrequency

Table 108 ETH Frequency

Name:	McuClockEthFrequency (ERAY frequency)		
Source:	This parameter is defined by IFX.		
File:	Mcu_PBCfg.c		
Range:	Default : 20 MHz		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	X	VariantPB
Description:	This parameter is purely used for reference purpose only. This is not applicable for TC23x/TC22x/TC21x platforms.		

SFRs Modified	None
Scope:	Driver
Dependency:	None

4.3.14 MCU Mode Setting Configuration

4.3.14.1 MCU Mode Settings

Table 109 McuMode

Name:	McuMode		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_PBCfg.c		
Range:	Values: 0: MCU_IDLE 1: MCU_SLEEP 2: MCU_STANDBY Default : 0		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All variants
Description:	It refers to the modes supported other than the RUN mode. (SLEEP mode, IDLE mode and STANDBY mode). The maximum number of modes configurable is 3. For a given ConfigSet of Mcu, there could be at the max 3 sets of modes. Mcu_SetMode will accept only for the configured modes. If Idle mode is only configured, Mcu_SetMode does not accept SLEEP/STANDBY command. However if Sleep or Standby mode is configured, SLEEP/STANDBY issues respective CPU(s) to Idle mode. The max number of modes configured becomes a part of the structure “Mcu_ConfigType”.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.14.2 MCU Idle Request Acknowledge Disable

Table 110 McuIdleReqAckSeqDisable

Name:	McuIdleReqAckSeqDisable
Source:	This parameter is defined by IFX
File:	Mcu_Cfg.h
Range:	Values: TRUE: To disable Idle request acknowledge sequence FALSE: Enable Idle request acknowledge sequence

	Default : FALSE		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is to disable Idle request acknowledge sequence issue to modules during transition to Sleep/Idle mode i.e. all peripherals intended to be inactive will be put in Sleep mode before the CPU/System goes to low power mode. This is applicable when user wants to enter to IDLE / SLEEP mode. When user enters to STANDBY mode, MCU driver will disable the IRADIS bit before entering Standby. #define MCU_DISABLE_IRADIS (STD_OFF/ON)		
SFRs Modified	SCU_PMSWCR1		
Scope:	Driver		
Dependency:	McuMode		

4.3.14.3 Container : MCU Standby Setting Configuration

MCU Driver multiplicity is 1:1.

The container McuStandbySettingConf and McuModeSettingConfig is to configure the standby mode related configuration for MCU Driver. Its multiplicity is 1:1. This container's parameters are enabled only when **McuMode** value is 2 (MCU_STANDBY).

4.3.14.3.1 Mcu Standby Pin Wakeup Enable

Table 111 Mcu<Pin>WakeUpEnable

Name:	Mcu<Pin>WakeUpEnable where Pin is ESR1, PinA and PinB		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: TRUE: Enable Wakeup from standby through pin <Pin> FALSE: Disable Wakeup from standby through pin <Pin> Default : FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All variants
Description:	This flag indicates if wakeup from Standby is possible through pin <Pin>. This gets configured as part of the structure "Mcu_StandbyModeType".		
SFRs Modified	SCU_PMSWCR0		
Scope:	Driver		
Dependency:	McuMode		

4.3.14.3.2 Mcu Standby Pin Digital Filter Enable

Table 112 Mcu<Pin>DigitalFilterEnable

Name:	Mcu<Pin>DigitalFilterEnable where Pin is ESR1, PinA, PinB and PORST
Source:	This parameter is defined by IFX
File:	Mcu_PBCfg.c

Range:	Values: TRUE: Enable digital filter for pin <Pin> FALSE: Enable digital filter for pin <Pin> Default : FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All variants
Description:	This flag indicates if digital filter for pin <Pin> should be enabled or not. This gets configured as part of the structure “Mcu_StandbyModeType”.		
SFRs Modified	SCU_PMSWCR0		
Scope:	Driver		
Dependency:	McuMode		

4.3.14.3.3 Mcu Standby Pin Edge Detection

Table 113 Mcu<Pin>EdgeDetection

Name:	Mcu<Pin>EdgeDetection where Pin is ESR1, PinA and PinB		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: TRIGGER_NONE: No trigger TRIGGER_RISING_EDGE: Trigger wakeup on rising edge of pin <Pin> TRIGGER_FALLING_EDGE: Trigger wakeup on falling edge of pin <Pin> TRIGGER_ANY_EDGE: Trigger wakeup on any edge of pin <Pin> Default : TRIGGER_ANY_EDGE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All variants
Description:	This flag indicates the edge trigger wakeup capability for pin <Pin>. This gets configured as part of the structure “Mcu_StandbyModeType”.		
SFRs Modified	SCU_PMSWCR0		
Scope:	Driver		
Dependency:	McuMode		

4.3.14.3.4 Mcu Standby ESR0 Tristate Enable

Table 114 McuESR0TristateEnable

Name:	McuESR0TristateEnable		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: TRUE: Enable ESR0 pin to be in tristate state FALSE: Disable ESR0 pin to be in tristate state Default : FALSE		
Configuration	Pre-compile	--	--

class	Link Time	--	--
	Post build	x	All variants
Description:	This flag indicates if ESR0 pin should be in tristate or not. All other pins except ESR1, PinA and PinB is put in Tristate before entering Standby mode by MCU driver. This gets configured as part of the structure “Mcu_StandbyModeType”.		
SFRs Modified	SCU_PMSWCR0		
Scope:	Driver		
Dependency:	McuMode		

4.3.14.3.5 Mcu Standby Ramp to Backup Frequency API Enable

Table 115 McuRampToBackupFreqApi

Name:	McuRampToBackupFreqApi		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: TRUE - Mcu_RampToBackUpClockFreq() API will be enabled FALSE - Mcu_RampToBackUpClockFreq() API will be disabled Default : FALSE		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the Mcu_RampToBackUpClockFreq() API which can ramp down the PLL clock from current configured frequency (done by Mcu_InitClock() API) to nearest value of backup frequency (100MHz). #define MCU_RAMP_TO_BACKUP_FREQ_API (STD_OFF/ON)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuMode		

4.3.14.3.6 Mcu Standby Set Control Register Runtime API

Table 116 McuSetStandbyWakeupControlApi

Name:	McuSetStandbyWakeupControlApi		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: TRUE - McuSetStandbyWakeupControlApi () API will be enabled FALSE - McuSetStandbyWakeupControlApi () API will be disabled Default : FALSE		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the McuSetStandbyWakeupControlApi() API which allows runtime update of SCU_PMSWCR0 register. This API can be used to enable/disable wakeup sources for Standby mode.		

	#define MCU_SET_STANDBY_CONTROL_API (STD_OFF/ON)
SFRs Modified	None
Scope:	Driver
Dependency:	McuMode

4.3.14.3.7 Mcu Standby CRC Calculation of Redundancy RAM region

Table 117 McuRARCrcCheckEnable

Name:	McuRARCrcCheckEnable		
Source:	This parameter is defined by IFX		
File:	Mcu_PBCfg.c		
Range:	Values: TRUE: CRC calculation will be done on RAM redundancy region of CPU0 DSPR FALSE: CRC calculation will not be done on RAM redundancy region of CPU0 DSPR Default : FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All variants
Description:	This flag indicates if CRC calculation should be done or not on RAM redundancy region of CPU0 DSPR i.e. 0x70002000 - 0x70002040. The 4 bytes CRC calculated value will stored immediately after RAM redundancy region (0x70002040). This gets configured as part of the structure “Mcu_StandbyModeType”.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuMode		

4.3.15 Container : McuDemEventParameterRefs

This container lists down all the Production error event configuration parameters for MCU Driver.

4.3.15.1 MCU_E_OSC_FAILURE

Table 118 MCU_E_OSC_FAILURE

Name:	MCU_E_OSC_FAILURE		
Source:	This parameter is defined by IFX		
File:	Mcu_Cfg.h		
Range:	NA		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the provision to enable or disable the Production error event related to Oscillator. A Production error is reported in case of oscillator failure while initializing the clock, if this parameter is enabled. #define MCU_E_OSC_FAILURE_DEM_REPORT (MCU_DISABLE_DEM_REPORT/MCU_ENABLE_DEM_REPORT) This parameter is applicable only for AS40.		

SFRs Modified	None
Scope:	Driver
Dependency:	None

4.3.15.2 MCU_E_CLOCK_FAILURE

Table 119 MCU_E_CLOCK_FAILURE

Name:	MCU_E_CLOCK_FAILURE		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu_Cfg.h		
Range:	NA		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the provision to enable or disable the Production error event related to PLL and Clocks. If clock source failure notification is enabled, then this parameter should be configured with appropriate DEM Id. #define MCU_E_CLOCK_FAILURE_DEM_REPORT (MCU_DISABLE_DEM_REPORT/MCU_ENABLE_DEM_REPORT) This parameter is applicable only for AS40.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuClockSourceFailureNotification		

4.3.15.3 MCU_E_ERAY_TIMEOUT

Table 120 MCU_E_ERAY_TIMEOUT

Name:	MCU_E_ERAY_TIMEOUT		
Source:	This parameter is defined by IFX		
File:	Mcu_Cfg.h		
Range:	NA		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the provision to enable or disable the Production error event related to ERAY PLL lock. #define MCU_E_ERAY_TIMEOUT_DEM_REPORT (MCU_DISABLE_DEM_REPORT/MCU_ENABLE_DEM_REPORT) This parameter is applicable only for AS40.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4 Container: McuPublishedInformation

Container holding all MCU specific published information parameters

4.4.1 McuResetReason Configuration

McuResetReasonConf container contains the configuration for the different type of reset reason that can be retrieved from Mcu_GetResetReason Api.

4.4.1.1 McuResetReason

The parameter represents different types of reset which a Microcontroller supports. There are 18 reset reasons available. The reset reasons and their values are given in the table below.

1. If XDM format is used, Mcu Preconfiguration file (McuPreconfiguration.xdm) can be used for configuring reset reasons. Mcu Preconfiguration file (McuPreconfiguration.xdm) has all the available reset reasons pre-configured in it.
2. If BMD format is used, then the reset reasons have to be manually configured by the user, under the McuResetReasonConf container.

Table 121 McuResetReason

Name:	McuResetReason		
Source:	This parameter is defined by AUTOSAR		
File:	Mcu.h		
Range:	Values: 0 to 255 MCU_ESR0_RESET=0x00, → ESR0 reset MCU_ESR1_RESET=0x01, → ESR1 reset MCU_SMU_RESET=0x02, → SMU reset MCU_SW_RESET=0x03, → Software reset MCU_STM0_RESET=0x04, → STM0 reset MCU_STM1_RESET=0x05, → STM1 reset MCU_STM2_RESET=0x06, → STM2 reset MCU_POWER_ON_RESET=0x07, → Power On reset MCU_CB0_RESET=0x08, → CB0 reset MCU_CB1_RESET=0x09, → CB1 reset MCU_CB3_RESET=0x0A, → CB3 reset MCU_TP_RESET=0x0B, → Tuning Protection reset MCU_EVR13_RESET=0x0C, → EVR13 Regulator Watchdog reset MCU_EVR33_RESET=0x0D, → EVR33 Regulator Watchdog reset MCU_SUPPLY_WDOG_RESET=0x0E, → Supply Watchdog reset MCU_STBYR_RESET=0x0F, → Standby Regulator Watchdog reset MCU_RESET_MULTIPLE=0xFEU → Multiple Reset reasons MCU_RESET_UNDEFINED=0xFFU → Reset is undefined		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	Values of different types of reset supported for AURIX HE family is listed in each of the container parameters. For AURIX EP family: MCU_STM1_RESET, MCU_STM2_RESET are not applicable		

SFRs Modified	None
Scope:	Driver
Dependency:	None

4.5 Container: CommonPublishedInformation

4.5.1 ArMajorVersion

Table 122 ArMajorVersion

Name:	ArMajorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	3		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the major version of the Autosar Specification.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.2 ArMinorVersion

Table 123 ArMinorVersion

Name:	ArMinorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	2		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the minor version of the Autosar Specification.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.3 ArPatchVersion

Table 124 ArPatchVersion

Name:	ArPatchVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	0		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the patch version of the Autosar Specification.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.4 SwMajorVersion

Table 125 SwMajorVersion

Name:	SwMajorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	Refer Mcu_Cfg.h		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the major version of the Software Driver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.5 SwMinorVersion

Table 126 SwMinorVersion

Name:	SwMinorVersion.
Source:	This parameter is defined by Autosar
File:	-
Default:	Refer Mcu_Cfg.h
Range:	0 - 65535

Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the minor version of the Software Driver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.6 SwPatchVersion

Table 127 SwPatchVersion

Name:	SwPatchVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	Refer Mcu_Cfg.h		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the patch version of the Software Driver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.7 VendorId

Table 128 VendorId

Name:	VendorId		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	17		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the VendorId		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.8 ModuleId

Table 129 ModuleId

Name:	ModuleId		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	101		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the VendorId		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.9 Release

Table 130 Release

Name:	Release		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	AURIX HE: _TRICORE_TC277 / _TRICORE_TC275 / _TRICORE_TC264 / _TRICORE_TC265 / _TRICORE_TC297 / _TRICORE_TC298 / _TRICORE_TC299 AURIX EP: _TRICORE_TC234 / _TRICORE_TC233 / _TRICORE_TC237 / _TRICORE_TC222 / _TRICORE_TC223 / _TRICORE_TC224 / _TRICORE_TC212 / _TRICORE_TC213 / _TRICORE_TC214		
Range:	NA		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the Release ID		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6 Container: Mcu Safety Configuration

This container contains the configuration parameters for the safety features of the MCU driver. The multiplicity is 1:1.

4.6.1 Mcu Safety Features Enable

Table 131 McuSafetyEnable

Syntax:	McuSafetyEnable

Source:	This parameter is defined by IFX		
File:	Mcu_Cfg.h		
Range:	Values: TRUE - Safety features enabled for MCU driver FALSE - Safety features disabled Default value: FALSE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety features of MCU driver. By enabling this flag, implicitly the following safety features are enabled: • Range check on API parameters will be enabled and it will be done by MCU driver. • In MCU driver, generated configuration structure will have a unique marker field which will be verified by the driver. The only dependency on marker generation is McuSafetyEnable configuration parameter. • Safety usage of MCU driver for user to set the respective MPU settings will be published in Mcu_ConfigDoc.html. (Mcu_ConfigDoc.html will be generated along with configuration). • Mcu_PerformReset() and Mcu_SetMode() APIs need explicit enabling of the write protection to write to the respective registers i.e. call to Mcal_ResetSafetyENDINIT_Timed() and Mcal_ResetENDINIT() respectively. #define MCU_SAFETY_ENABLE (STD_OFF/STD_ON)		
SFRs Modified	SCU_CCUCON3, SCU_CCUCON4, SCU_CCUCON9		
Scope:	Driver.		
Dependency:	None		

4.6.2 Mcu Initialization Check

Table 132 McuInitCheckApi

Syntax:	McuInitCheckApi		
Source:	This parameter is defined by IFX		
File:	Mcu_Cfg.h		
Range:	Values: TRUE - Mcu_InitCheck() API will be enabled FALSE - Mcu_InitCheck() API will be disabled Default value: TRUE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety feature Mcu_InitCheck() which verifies the initialization done by MCU driver. #define MCU_INITCHECK_API (STD_OFF/ON) This #define will not be generated if McuSafetyEnable is FALSE.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuSafetyEnable		

4.6.3 Mcu Clock Monitoring Enable

Table 133 McuClockMonitoringEnable

Syntax:	McuClockMonitoringEnable		
Source:	This parameter is defined by IFX		
File:	Mcu_Cfg.h		
Range:	Values: TRUE - Clock monitoring safety feature will be enabled FALSE - Clock monitoring safety feature will be disabled Default value: TRUE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the clock monitoring safety feature. #define MCU_ENABLE_CLOCK_MONITORING (STD_OFF/ON)		
SFRs Modified	SCU_CCUCON3, SCU_CCUCON4, SCU_CCUCON9		
Scope:	Driver		
Dependency:	McuSafetyEnable		

4.6.4 Mcu Get Mode

Table 134 McuGetModeApi

Syntax:	McuGetModeApi		
Source:	This parameter is defined by IFX		
File:	Mcu_Cfg.h		
Range:	Values: TRUE - Mcu_GetMode() API will be enabled FALSE - Mcu_GetMode() API will be disabled Default value: TRUE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety feature Mcu_GetMode() which verifies the power management state of the CPU. #define MCU_GETMODE_API (STD_OFF/ON) This #define will not be generated if McuSafetyEnable is FALSE.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuSafetyEnable		

4.7 Derived Configuration Parameters

These parameters represent the additional information required by the Driver and are automatically calculated from the tool.

4.7.1 Number of MCU Mode Settings

Table 135 Number of MCU Mode settings

Syntax:	MCU_MAX_NO_MODES		
Source:	This parameter is AUTOSAR specific		
File:	Mcu_Cfg.h		
Range:	Values: 3		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	The Maximum number of modes that the user is allowed to configure is restricted to 3 as there are only three modes, other than run mode, supported by AURIX #define MCU_NUMBER_OF_MODES (<Count>) Refer **McuMode** in this document.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuModeSetting		

4.7.2 Number of Clock Settings

The Maximum number of clock settings that the user has configured.

Table 136 Number of MCU Clock settings

Syntax:	MCU_NUMBER_OF_CLOCK_SETTINGS		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: 1 - *		
Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	It refers to the number of clock settings that are configured by the user. #define MCU_NUMBER_OF_CLOCK_SETTINGS ((Mcu_ClockType)(<count>))		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuClockSettingConf		

4.7.3 Number of Ram Settings

The Maximum number of Ram settings that the user has configured.

Table 137 Number of Ram settings

Syntax:	MCU_RAM_SECTORS
Source:	This parameter is driver specific
File:	Mcu_Cfg.h
Range:	Values: 0 - *

Configuration class	Pre-compile	x	All variants
	Link Time	--	--
	Post build	--	--
Description:	It refers to the number of Ram settings that are configured by the user. #define MCU_RAM_SECTORS (<Count>U)		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuRamSectorSettingConf		

4.7.4 Gtm availability

Table 138 Gtm availability

Syntax:	MCU_GTM_USED		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Whenever Gtm is configured, this parameter becomes STD_ON. If there are no elements under GtmConfiguration, then this parameter becomes STD_OFF. #define MCU_GTM_USED (<Gtm availability>) Note: In PB selectable setup, when Gtm is configured in one config set of Mcu, please ensure that all the config sets contain Gtm. Else, a configuration generation error is flashed.		
SFRs Modified	Refer **4.3.8** for more details.		
Scope:	Driver		
Dependency:	GtmConfiguration		

4.7.5 DMA availability

Table 139 DMA availability

Syntax:	MCU_DMA_USED		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Whenever MCU DMA container is configured, this parameter becomes STD_ON. If there are no elements under DmaConfiguration, then this parameter becomes STD_OFF.		

	#define MCU_DMA_USED (<DMA availability>)
SFRs Modified	Refer **4.3.7** for more details.
Scope:	Driver
Dependency:	DmaConfiguration

4.7.6 CRC HW availability

Table 140 CRC HW availability

Syntax:	MCU_CRC_HW_USED		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Whenever CRC plugin exists and CRC HW is configured, this parameter becomes STD_ON. If not, then this parameter becomes STD_OFF. #define MCU_CRC_HW_USED (<CRC HW availability>) This will be STD_OFF for AURIX EP platforms since FCE engine is not available.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	Crc plugin		

4.7.7 CRC 8-bit HW availability

Table 141 CRC 8-bit HW availability

Syntax:	MCU_CRC8_HW_MODE		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Whenever MCU_CRC_HW_USED is ON and CRC 8-bit HW is configured, this parameter becomes STD_ON. If not, then this parameter is not generated. #define MCU_CRC8_HW_MODE (<CRC 8-bit HW availability>) This is not applicable for AURIX EP platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	Crc plugin		

4.7.8 CRC 16-bit HW availability

Table 142 CRC 16-bit HW availability

Syntax:	MCU_CRC16_HW_MODE		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Whenever MCU_CRC_HW_USED is ON and CRC 16-bit HW is configured, this parameter becomes STD_ON. If not, then this parameter is not generated. #define MCU_CRC16_HW_MODE (<CRC 16-bit HW availability>) This is not applicable for AURIX EP platforms.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	Crc plugin		

4.7.9 FM PLL Enable

Table 143 FM PLL Enable

Syntax:	MCU_FMPLL_ENABLE		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Switch to enable/disable PLL frequency modulation. #define MCU_FMPLL_ENABLE (STD_ON/STD_OFF)		
SFRs Modified	SCU_PLLCON0		
Scope:	Driver		
Dependency:	McuFmPllEnable		

4.7.10 Disable ERAY PLL

Table 144 Disable ERAY PLL

Syntax:	MCU_DISABLE_ERAY_PLL		
Source:	This parameter is driver specific		
File:	Mcu_Cfg.h		
Range:	Values: STD_ON or STD_OFF		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--

	Post build	--	--
Description:	Switch to enable/disable ERAY PLL. #define MCU_DISABLE_ERAY_PLL (STD_ON/STD_OFF) This macro becomes STD_ON only if McuMainOscillatorFrequency < 16 MHz. This macro will prevent initialization of ERAY PLL and ensure ERAY PLL is in power down mode.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuMainOscillatorFrequency		

These derived parameters are **precompile** parameters and hence the parameters in the “Dependency” field should not be modified after the build process

5 Published Parameters

This section describes the parameters published by the MCU driver.

INFO: Due to an internal issue in Tresos 2012.a, sub-containers are not visible. However, these values are present in the configured Mcu XDM/EPC.

5.1 VendorId

Table 145 VendorId

Syntax:	MCU_VENDOR_ID
Type:	Macro - #define
File:	Mcu_Cfg.h
Value:	17
Description:	Vendor is Infineon Technologies

5.2 ModuleId

Table 146 ModuleId

Syntax:	MCU_MODULE_ID
Type:	Macro - #define, type casted to uint16
File:	Mcu_Cfg.h
Value:	101 (0x65)
Description:	This macro gives the Mcu driver module ID as described by AUTOSAR

5.3 InstanceId

Table 147 InstanceId

Syntax:	MCU_INSTANCE_ID
Type:	Macro - #define, type casted to uint16
File:	Mcu_Cfg.h
Value:	0

Description:	This macro gives the Mcu driver Insatance ID as described by AUTOSAR

5.4 SwMajorVersion

Table 148 SwMajorVersion

Syntax:	MCU_SW_MAJOR_VERSION
Type:	Macro – #define
File:	Mcu_Cfg.h
Value:	Refer Mcu_Cfg.h
Description:	Major version number of the vendor specific implementation of the driver. The numbering is vendor specific. The MAJOR_VERSION is incremented if the driver is not downwards compatible any more (e.g. existing API changed)

5.5 SwMinorVersion

Table 149 SwMinorVersion

Syntax:	MCU_SW_MINOR_VERSION
Type:	Macro - #define
File:	Mcu_Cfg.h
Value:	Refer Mcu_Cfg.h
Description:	Minor version number of the vendor specific implementation of the driver. The numbering is vendor specific. MINOR_VERSION is incremented if the driver is still downwards compatible (e.g. new functionality added)

5.6 SwPatchVersion

Table 150 SwPatchVersion

Syntax:	MCU_SW_PATCH_VERSION
Type:	Macro - #define
File:	Mcu_Cfg.h
Value:	Refer Mcu_Cfg.h
Description:	Patch level version number of the vendor specific implementation of the driver. The numbering is vendor specific. The PATCH_VERSION is incremented if the driver is still upwards and downwards compatible (e.g. bug fixed)

5.7 ARMajorVersion

Table 151 ARMajorVersion

Syntax:	MCU_AR_RELEASE_MAJOR_VERSION
Type:	Macro - #define
File:	Mcu_Cfg.h

Syntax:	MCU_AR_RELEASE_MAJOR_VERSION
Value:	Refer Mcu_Cfg.h
Description:	Major version number of the AUTOSAR specification.

5.8 ARMinorVersion

Table 152 ARMinorVersion

Syntax:	MCU_AR_RELEASE_MINOR_VERSION
Type:	Macro - #define
File:	Mcu_Cfg.h
Value:	Refer Mcu_Cfg.h
Description:	Minor version number of the AUTOSAR specification.

5.9 ARPatchVersion

Table 153 ARPatchVersion

Syntax:	MCU_AR_RELEASE_REVISION_VERSION
Type:	Macro - #define
File:	Mcu_Cfg.h
Value:	Refer Mcu_Cfg.h
Description:	Patch version number of the AUTOSAR specification.

6 API Documentation

6.1 API Type definitions

6.1.1 Type Definition Mcu_ConfigType

Table 154 Type definition Mcu_ConfigType

Syntax:	Mcu_ConfigType	
Type:	Struct	
File:	Mcu.h	
Range: Mcu_ConfigType:	This is datatype is used for the configuration of the entire MCU driver	
	const uint32 Marker	Holds the module id and instance id information; required only when MCU_SAFETY_ENABLE is STD_ON
	const Mcu_ClockCfgType *Mcu_ClockCfgPtr	Pointer to Clock setting config structure

	const Mcu_RamCfgType *Mcu_RamCfgPtr	Pointer to RAM Section config structure
	const GtmConfigType *Gtm_ConfigData	Ptr to Gtm Configuration structure. This is included in the structure if GTM_USED is ON
	uint32 Mcu_ResetCfg	Reset request configuration is included in the structure if MCU_PERFORM_RESET_API is ON
	Mcu_ClockType NoOfClockCfg	Total number of Clock Settings
	Mcu_RamSectionType NoOfRamCfg	Total number of RAM Sectors
	Mcu_ModeType MaxMode	Total number of modes that can be switched in the given ConfigSet.
	const Mcu_StandbyModeType *StandbyCfgPtr	Pointer to Standby mode configuration structure
Description	Type to define the MCU related configuration VariantPB: This typedef holds the Post build Configuration Parameters. A variable of this type is directly referenced by the Mcu driver implementation. Only a pointer to this config structure shall be passed to the initialization function.	

6.1.2 Type Definition Mcu_ClockCfgType

Table 155 Type Definition Mcu_ClockCfgType

Syntax:	Mcu_ClockCfgType		
Type:	Struct		
File:	Mcu.h		
Range:	uint8 K2div[8]	K2 divider step values are stored. Valid values are 0 – 127 Even values are preferred to get 50% duty cycle	
	uint32 K2RampToPllDelayTicks [8]	Delay ticks for ramping up the K2 div are calculated and rounded of as per K2div values.	
	uint32 K2RampToBackUpDelayTicks [8]	Delay ticks for ramping down the K2 div are calculated and rounded of as per K2div values. Required only when required only when MCU_RAMP_TO_BACKUP_FREQ_API is STD_ON.	
	Mcu_ClockDivValues (Struct)	unsigned_int K1div:7	K1DIV: K1 Divider value (Even values are preferred to get 50% duty cycle.)
		unsigned_int Ndiv:7	N Divider value
		unsigned_int Pdiv:4	P Divider value
		unsigned_int K3div:7	K3 Divider Value
		unsigned_int K2Steps:4	Number of Intermediate Steps
		unsigned_int PllMode:1	• Normal Mode - Bypassing K1 divider • Prescaler Mode – Bypassing P, N, K2, K3 dividers
		unsigned_int Reserved:2	Padding bits to align to 32- bit boundary
	Mcu_ErayPllDiv	unsigned_int	N Divider Value

	Values (Struct)	McuErayNDivider:5	
		unsigned_int McuErayK2Divider:7	K2 Divider Value
		unsigned_int McuErayK3Divider:7	K3 Divider Value
		unsigned_int McuErayPDivider:4	P Divider Value
		unsigned_int McuErayPDivider:9	Padding bits to align to 32- bit boundary
	uint32 Ccucon0	Ccucon0 made up of Baud1Divider, Baud2Divider, SRIDivider, SPBDivider, FSI2Divider, FSIDivider and ClockSelectADC,	
	uint32 Ccucon1	Ccucon1 made up of CANDivider, Eray Divider, STM Divider, GTM Divider, ETH Divider, ASCLINS Divider and ASCLINF Divider	
	uint32 Ccucon2	Ccucon2 is made up of BBB Divider	
	uint32 Ccucon5	Ccucon5 is made up of MAX Divider Also EBU Divider if TC29x	
	uint32 Ccucon6	Ccucon6 is made up of CPU0 Divider	
	uint32 Ccucon7	Ccucon7 is made up of CPU1 Divider	
	uint32 Ccucon8	Ccucon8 is made up of CPU2 Divider	
	uint32 Ccucon3	Ccucon3 is made up of clock monitor divider values for PLL, PLL Eray and SRI; required only when MCU_SAFETY_ENABLE is STD_ON	
	uint32 Ccucon4	Ccucon4 is made up of clock monitor divider values for GTM, STM and SPB; required only when MCU_SAFETY_ENABLE is STD_ON	
	uint32 Ccucon9	Ccucon9 is made up of clock monitor divider values for ADC; required only when MCU_SAFETY_ENABLE is STD_ON	
	uint32 Pllcon2	Pllcon2 is used to configure the modulation amplitude for FM PLL settings; required only when MCU_FMPLL_ENABLE is STD_ON	
	uint8 K2RampToPllDelayConf	K2RampToPllDelayConf denotes the McuK2DivRampToPllConfDelay value configured in micro-second.	
Description:	Type to define the configuration structure to configure MCU Clock setting.		

6.1.3 Type Definition Mcu_ClockType

Table 156 Type Definition Mcu_ClockType

Syntax :	Mcu_ClockType
Type :	uint32
File :	Mcu.h
Range :	Range of uint32 (0-0xFFFFFFFF)
Description :	Identification for clock setting, which is configured in the configuration structure

6.1.4 Type Definition Mcu_ModeType

Table 157 Type Definition Mcu_ModeType

Syntax :	Mcu_ModeType
Type :	uint32
File :	Mcu.h
Range :	Range of uint32 (0-0xFFFFFFFF)
Description :	Identification for MCU mode, which is configured in the configuration structure

6.1.5 Type Definition Mcu_ModeStateType

Table 158 Type Definition Mcu_ModeStateType

Syntax :	Mcu_ModeStateType
Type :	enum
File :	Mcu.h
Range :	MCU_UNDEFINED_MODE = 0x0U /* CPU undefined mode */ MCU_NORMAL_RUN_MODE = 0x1U /* CPU running in normal mode */ MCU_CPUIDLE_REQ_MODE = 0x2U /* CPU Idle mode requested */ MCU_CPUIDLE_ACK_MODE = 0x3U /* CPU Idle mode acknowledged */ MCU_SLEEP_REQ_MODE = 0x4U /* CPU Sleep mode requested */ MCU_STANDBY_REQ_MODE = 0x6U /* CPU Standby mode requested */
Description :	MCU power mode state for the respective core

6.1.6 Type Definition Mcu_RamBaseAdrType

Table 159 Type Definition Mcu_RamBaseAdrType

Syntax :	Mcu_RamBaseAdrType
Type :	void *
File :	Mcu.h
Range :	Pointer to an address (0-0xFFFFFFFF)
Description :	RAM Section base address

6.1.7 Type Definition Mcu_RamSizeType

Table 160 Type Definition Mcu_RamSizeType

Syntax :	Mcu_RamSizeType
Type :	uint32
File :	Mcu.h
Range :	Range of uint32 (0-0xFFFFFFFF)
Description :	Ram Section Size

6.1.8 Type Definition Mcu_RamPrstDatType

Table 161 Type Definition Mcu_RamPrstDatType

Syntax :	Mcu_RamPrstDatType
Type :	uint8
File :	Mcu.h
Range :	Range of uint8
Description :	Data Pre-setting to be initialized

6.1.9 Type Definition Mcu_RamCfgType

Table 162 Type Definition Mcu_RamCfgType

Syntax :	Mcu_RamCfgType	
Type :	Struct	
File :	Mcu.h	
Range :	Mcu_RamBaseAdrType	Start address of the RAM Section
	Mcu_RamSizeType	Size of the RAM to be initialized
	Mcu_RamPrstDatType	Preset value to be written in all locations
Description :	Identification for RAM section setting.	

6.1.10 Type Definition Mcu_ModeStateType

Table 163 Type Definition Mcu_ModeStateType

Syntax :	Mcu_ModeStateType	
Type :	enumeration	
File :	Mcu.h	
Range :	MCU_UNDEFINED_MODE	CPU undefined mode
	MCU_NORMAL_RUN_MODE	CPU running in normal mode
	MCU_CPUIDLE_REQ_MODE	CPU Idle mode requested
	MCU_CPUIDLE_ACK_MODE	CPU Idle mode acknowledged
	MCU_SLEEP_REQ_MODE	CPU Sleep mode requested
	MCU_STANDBY_REQ_MODE	CPU Standby mode requested
Description:	Status of the power management mode each CPU is currently in.	

6.1.11 Type Definition Mcu_RamStateType

Table 164 Type Definition Mcu_RamStateType

Syntax :	Mcu_RamStateType	
Type :	Enumeration	
File :	Mcu.h	
Range :	MCU_RAMSTATE_INVALID	Ram content is not valid or unknown (default).
	MCU_RAMSTATE_VALID	Ram content is valid
Description :	This type provides the validity status of RAM.	

6.1.12 Type Definition Mcu_RawResetType

Table 165 Type Definition Mcu_RawResetType

Syntax :	Mcu_RawResetType
Type :	Enumeration
File :	Mcu.h
Range :	32 bit value of Reset Status registers (SCU_RSTSTAT).
Description :	This type specifies the reset reason in raw register format read from a reset status register.

6.1.13 Type Definition Mcu_PllStatusType

Table 166 Type Definition Mcu_PllStatusType

Syntax :	Mcu_PllStatusType
Type :	Enumeration
File :	Mcu.h
Range :	MCU_PLL_LOCKED :PLL is locked MCU_PLL_UNLOCKED :PLL is unlocked MCU_PLL_STATUS_UNDEFINED :PLL status is unknown
Description :	This is a status value returned by the API service Mcu_GetPllStatus () of MCU driver.

6.1.14 Type Definition Mcu_ResetType

Table 167 Type Definition Mcu_ResetType

Syntax :	Mcu_ResetType
Type :	Enumeration
File :	Mcu.h
Range :	MCU_ESR0_RESET = 0 MCU_ESR1_RESET MCU_SMU_RESET MCU_SW_RESET MCU_STM0_RESET MCU_STM1_RESET MCU_STM2_RESET MCU_POWER_ON_RESET MCU_CB0_RESET MCU_CB1_RESET MCU_CB3_RESET MCU_TP_RESET MCU_EVR13_RESET MCU_EVR33_RESET MCU_SUPPLY_WDOG_RESET MCU_STBYR_RESET MCU_RESET_MULTIPLE

	MCU_RESET_UNDEFINED
Description :	Reset source types in MCU.

6.1.15 Type Definition Mcu_StandbyModeType

Table 168 Type Definition Mcu_StandbyModeType

Syntax :	Mcu_StandbyModeType	
Type :	Struct	
File :	Mcu.h	
Range :	uint32	PMSWCR0 register value
	uint8	True/False to do CRC calculation on RAM redundancy region
Description :	Type to define the standby mode configuration structure.	

6.2 API Functions

This chapter describes the API interfaces provided by MCU driver.

6.2.1 Mcu_Init

Table 169 Specification of API Mcu_Init

Service name:	Mcu_Init	
Syntax:	void Mcu_Init (Const Mcu_ConfigType *ConfigPtr)	
Service ID:	0x00	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non-reentrant	
Parameters (in):	ConfigPtr	Pointer to the configuration structure
Parameters (out):	None	
Return value:	None	
Description:	This routine initializes the MCU driver. The intention of this function is to make the configuration settings for clock and RAM sections visible within the MCU driver. It also stores the reset register to determine the reset reason. It also sets the power management registers according to the configuration. It also initializes the GTM by calling Gtm_Init(), if MCU_GTM_USED is ON. It also initializes the DMA clock if MCU_DMA_USED is ON. It also initializes the FCE clock and respective FCE configurations (8-bit kernel, 16-bit kernel and 32-bit kernel) if MCU_CRC_HW_USED is ON. After execution of Mcu_Init() the configuration data is accessible and can be used from the Mcu services like Mcu_InitRamSection(),Mcu_InitClock() etc.	
Caveats:	• This function shall be called from CPU Core 0 only. • If CLC initialization of GTM, DMA or FCE fails, the Mcu_Driver_State variable is not set to Initialized.	

Configuration:	None
DET:	MCU_E_PARAM_CONFIG (10) : When passed configuration pointer parameter is a NULL pointer
DEM:	No for this API.
Safety Error reporting	SafeMcal_ReportError is invoked for following errors: • MCU_E_PARAM_CONFIG (10): When passed configuration pointer parameter is a NULL pointer or passed pointer is not the desired pointer when MCU_PB_FIXEDADDR is ON or Marker value is not matching.
Implementation Comments	If safety is enabled, plausibility check of ConfigPtr is done to verify if the pointer is valid. This function initializes the MCU configuration with the user configured values. This function call initializes GTM, DMA and FCE if configured.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'False'.

6.2.2 Mcu_InitRamSection

Table 170 Specification of API

Service name:	Mcu_InitRamSection	
Syntax:	Std_ReturnType Mcu_InitRamSection (Mcu_RamSectionType RamSection)	
Service ID:	0x01	
Sync/Async:	Synchronous	
Origin	Autosar	
Reentrancy:	Non-reentrant	
Parameters (in):	RamSection	Index of RAM section configuration instances
Parameters (out):	None	
Return value:	E_OK: Command has been accepted E_NOT_OK: Command has not been accepted e.g. due to parameter error	
Description:	This function initializes the RAM, section wise. The definition of the section and the initialization value is provided from the configuration structure.	
Caveats:	This function requires an execution of Mcu_Init () before it can be used.	
Configuration:	None	
DET:	• MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before Mcu_InitRamSection () can be used. • MCU_E_PARAM_RAMSECTION (13) is reported when passed parameter RamSection is not within the sections defined in the configuration data structure.	
DEM:	No DEM for this API	

Safety Error reporting	SafeMcal_ReportError is invoked for following errors: • MCU_E_PARAM_RAMSECTION (13): when passed parameter RamSection is not within the sections defined in the configuration data structure.
Implementation Comments	This function initializes the specified RAM Section (pointed by RamSectionBaseAddr) with the constant data referred by configured preset data. The MCU module's environment shall call the function Mcu_InitRamSection only after the MCU module has been initialized using the function Mcu_Init.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'False'.

6.2.3 Mcu_InitClock

Table 171 Specification of API Mcu_InitClock

Service name:	Mcu_InitClock	
Syntax:	Std_ReturnType Mcu_InitClock (Mcu_ClockType ClockSetting)	
Service ID:	0x02	
Sync/Async:	Synchronous	
Origin	Autosar	
Reentrancy:	Non-reentrant	
Parameters (in):	ClockSetting	Index of Clock section configuration instances
Parameters (out):	None	
Return value:	E_OK: command has been accepted E_NOT_OK: command has not been accepted	
Description:	This function initializes the PLL, ERAYPPLL and other MCU specific clock options. The clock configuration parameters (P, N, K3 and the initial K2 divider values for both the PLL's) are provided via the configuration structure. In this function the PLL lock procedure is started (if PLL is initialized) but the function does not wait until the PLL is locked. In case the Eray PLL is not locked, DEM is reported (if enabled).	
Caveats:	• This function shall be called from CPU Core 0 only. • This function requires an execution of Mcu_Init () before it can be used. .	
Configuration:	None	
DET:	• MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before this API can be used. • MCU_E_PARAM_CLOCK (11) is reported when passed parameter (ID) ClockSetting is not within the range of the defined configuration.	
DEM:	McuErayDemTimeoutId (If Eray Pll not locked).	

	McuOscFailureDemId (If oscillator fails during initialization)
Safety Error reporting	SafeMcal_ReportError is invoked for following errors: • MCU_E_PARAM_CLOCK (11): when passed parameter ClockSetting is not within the range of the defined configuration.
Implementation Comments	No support for clock unit to run solely on back-up clock, only PLL mode supported. There are two possible PLL modes namely: • NORMAL NODE • PRESCALER MODE For Normal Mode After Mcu_InitClock() is executed, user has to wait for SCU_PLLSTAT.VCOLOCK bit to be set (by polling using Mcu_GetPllStatus()) and then activate distribution of the PLL clock (by executing Mcu_DistributePllClock() API which will disable VCO Bypass Mode) For Prescaler Mode The user need not call any additional APIs. Mcu_DistributePllClock () should not be called by the user if clock mode is set to Prescaler mode. If clock monitoring feature is enabled, clock monitoring registers will be enabled for prescaler mode. The MCU module's environment shall only call the function Mcu_InitClock after the MCU module has been initialized using the function Mcu_Init. If current clock mode is normal mode, Mcu_InitClock will ramp the current frequency close to Backup Clock frequency, switches the clock source to backup clock and then configures the PLL dividers. If current clock mode is prescaler mode this api will directly switch the clock source to backup clock and ramps to the desired configured frequency. Mcu_InitClock and Mcu_DistributePllClock should be called before SMU is put in run state. Unexpected SMU alarms may trigger when MCU is configuring the clocks (clock dividers). Hence, above MCU API's temporarily disable the SMU alarms related to OSC and CLOCK, and restore the status back at the end of the API.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'False'.

6.2.4 Mcu_DistributePllClock

Table 172 Specification of API Mcu_DistributePllClock

Service name:	Mcu_DistributePllClock
Syntax:	void Mcu_DistributePllClock (void

	)
Service ID:	0x03
Sync/Async:	Synchronous
Origin	Autosar
Reentrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This function activates the distribution of PLL clock as System Clock. It also de-activates the VCO as the PLL is already locked. This function switches clock source to PLL and then updates the subsequent K2 divider steps in normal mode. Before each update a delay of 6 current CPU cycles is awaited. If this function is not executed, configured PLL clock frequency will not be distributed to the system.
Caveats:	• This function shall be called from CPU Core 0 only. • This function has to be called only after the status of the PLL is detected as locked with Mcu_GetPllStatus. • This function requires an execution of Mcu_Init () before it can be used.Before executing this API, all the PLL parameters have to be set (Mcu_InitClock should be executed successfully (return value is E_OK)). • This function should not be executed if clock mode is set to Pre-scalar mode. • All individual PWM/ICU/GPT modules shall be invoked by the user after PLL setup only.
Configuration:	None
DET:	• MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before this API can be used. • MCU_E_PLL_NOT_LOCKED (14) is reported if the status of the PLL is not LOCKED.
DEM:	No DEM for this API.
Safety Error reporting	SafeMcal_ReportError is invoked for following errors: • MCU_E_PLL_NOT_LOCKED (14): if the status of the PLL is not LOCKED.
Implementation Comments:	The MCU module's environment shall only call the function Mcu_DistributePllClock after the status of the PLL has been detected as locked by the function Mcu_GetPllStatus. The MCU module's environment shall execute the function Mcu_DistributePllClock to activate the PLL clock after the PLL is locked. If clock monitoring feature is enabled, clock monitoring registers are enabled by Mcu driver. Mcu_InitClock and Mcu_DistributePllClock should be called before SMU is put in run state. Unexpected SMU alarms may trigger when MCU is configuring the clocks (clock dividers). Hence, above MCU API's temporarily disable the SMU alarms related to OSC and CLOCK, and restore the status back at the end of the API.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided

	configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeInitApiEnable is ‘False’.

6.2.5 Mcu_GetPllStatus

Table 173 Specification of API Mcu_GetPllStatus

Service name:	Mcu_GetPllStatus
Syntax:	Mcu_PllStatusType Mcu_GetPllStatus (void)
Service ID:	0x04
Sync/Async:	Synchronous
Origin	Autosar
Reentrancy:	Reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	Mcu_PllStatusType
Description:	This function provides the lock status of the PLL.
Caveats:	This function requires an execution of Mcu_Init () before it can be used.
Configuration:	None
DET:	None
DEM:	No DEM for this API
Safety Error reporting	None
Implementation Comments	This function reads the value of VCOLOCK bit field in SCU_PLLSTAT register.(AURIX Special function register) • MCU_E_UNINIT is reported when Mcu_Init () is not executed before this API can be used and MCU_PLL_STATUS_UNDEFINED is returned.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeInitApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeInitApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeInitApiEnable is ‘False’.

6.2.6 Mcu_GetResetReason

Table 174 Specification of API Mcu_GetResetReason

Service name:	Mcu_GetResetReason
Syntax:	Mcu_ResetType Mcu_GetResetReason (void)
Service ID:	0x05

Sync/Async:	Synchronous
Origin	Autosar
Reentrancy:	Reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	Mcu_ResetType
Description:	The function reads the reset type from the hardware. (AURIX special function register SCU_RSTSTAT)
Caveats:	This function requires an execution of Mcu_Init () before it can be used.
Configuration:	None
DET:	MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before this API and MCU_RESET_UNDEFINED is returned.
DEM:	No DEM for this API
Safety Error reporting	None
Implementation Comments	None.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.

6.2.7 Mcu_GetResetRawValue

Table 175 Specification of API Mcu_GetResetRawValue

Service name:	Mcu_GetResetRawValue
Syntax:	Mcu_RawResetType Mcu_GetResetRawValue (void)
Service ID:	0x06
Sync/Async:	Synchronous
Origin	Autosar
Reentrancy:	Reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	Mcu_RawResetType
Description:	The function returns the reset status register value.
Caveats:	This function requires an execution of Mcu_Init () before it can be used.
Configuration:	None
DET:	MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before this API and the register value is returned.

DEM:	No DEM for this API
Safety Error reporting	None
Implementation Comments	32 bit content of reset status register is returned. (AURIX special function register SCU_RSTSTAT)
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'False'.

6.2.8 Mcu_PerformReset

Table 176 Specification of API Mcu_PerformReset

Service name:	Mcu_PerformReset
Syntax:	void Mcu_PerformReset (void)
Service ID:	0x07
Sync/Async:	Synchronous
Origin	Autosar
Reentrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	The function performs a microcontroller reset. The MCU specific reset type to be performed by this service should be configured in the reset configuration set. Refer section **4.3.1**
Caveats:	This function requires an execution of Mcu_Init () before it can be used.
Configuration:	This service is enabled only when the preprocessor switch MCU_PERFORM_RESET_API is defined to STD_ON.
DET:	MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before using this API.
DEM:	No DEM for this API.
Safety Error reporting	None

Implementation Comments	A soft reset is invoked by writing to Reset Request register SCU_SWRSTCON register. The soft reset can exclude/include System Timer Refer section **4.3.1** The MCU module's environment shall only call the function Mcu_PerformReset after the MCU module has been initialized by the function Mcu_Init. If MCU_SAFETY_ENABLE is ON, then this function shall not invoke Mcal_ResetSafetyENDINIT_Timed to access SCU_SWRSTCON register. If MCU_SAFETY_ENABLE is ON, then User has to call Mcal_ResetSafetyENDINIT_Timed() before calling Mcu_PerformReset(). (Refer MCALLIB user manual for Mcal_ResetSafetyENDINIT_Timed api description **[10]).**
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'False'.

6.2.9 Mcu_SetMode

Table 177 Specification of API Mcu_SetMode

Service name:	Mcu_SetMode
Syntax:	void Mcu_SetMode (Mcu_ModeType McuMode)
Service ID:	0x08
Sync/Async:	Synchronous
Origin	Autosar
Reentrancy:	Non-reentrant
Parameters (in):	McuMode
Parameters (out):	None
Return value:	None
Description:	This function activates the MCU power down modes. McuSystemModeCpuCore decides which core can trigger Sytem Modes McuIdleModeCpuCore decides which core can trigger Idle Mode.
Caveats:	This function requires an execution of Mcu_Init () before it can be used.
Configuration:	None
DET:	• MCU_E_UNINIT (15) is reported if Mcu_Init () was not executed before calling this API. • MCU_E_PARAM_MODE (12) is reported when an invalid McuMode is passed.
DEM:	No DEM for this API.
Safety Error reporting	SafeMcal_ReportError is invoked for following errors: • MCU_E_PARAM_MODE (12): when an invalid McuMode parameter is passed

Implementation Comments	If MCU_SAFETY_ENABLE is ON, then this function does not invoke Mcal_ResetENDINIT/ Mcal_ResetSafetyENDINIT to access power management registers. It is the user's responsibility to enable the write protection to access power management registers. • The MCU module's environment shall only call the function Mcu_SetMode after the MCU module has been initialized by the function Mcu_Init. Note: The environment of the function Mcu_SetMode has to ensure that the ECU is ready for reduced power mode activation • The API Mcu_SetMode assumes that all interrupts are disabled prior the call of the API by the calling instance. The implementation has to take care that no wakeup event is lost. This could be achieved by a check whether pending wakeup interrupts already have occurred even if Mcu_SetMode has not set the controller to power down mode yet. The wakeup events should be cleared before invoking Mcu_SetMode. • Before achieving any low power down mode, Mcu_SetMode verifies and allows only if the current executing CPU has the permission to enter the desired low power mode. • For Standby mode, Mcu_SetMode does the following sequence: o Verifies and allows only if the current executing CPU has the permission to enter the desired low power mode. This check is not present in AURIX EP platforms. o Handle the Standby RAM redundancy data to help the firmware on wake-up from standby o If configured, CRC calculation of the Standby RAM redundancy region. Store the 4 byte CRC value immediately after the RAM redundancy region. o If enabled, disable the clock monitoring unit o Select the backup clock as source for all clocks o Disable Oscillator and put PLL/ERAY_PLL to power saving mode o Set IRADIS bit in SCU_PMSWCR1 to 1 to disable Idle Request Acknowledge Sequence. o Set pads to Tristate state and enable CPU0 DSPR as the Standby RAM. o Write to REQSLP with Standby mode value to the power management register Note: User has to cover all the remaining applicable pre-requisites necessary to enter Standby mode as mentioned in UM **[3],[6],[7]** • Before entering Standby mode, it is the user's responsibility to back up necessary system relevant information to Standby RAM (CPU0 DSPR). This is because during Standby only Standby RAM is powered and wakeup from Standby results in a warm reset. Also, 0x70002000 to 0x70002044 (including 4 bytes of CRC) should be reserved in CPU0 DSPR, since this area is used by boot-up firmware for handling RAM redundancy data. • Refer **Figure 7** for SLEEP/STANDBY mode entry sequence
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided

	configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.

6.2.10 Mcu_GetRamState

Table 178 Specification of API Mcu_GetRamState

Service name:	Mcu_GetRamState	
Syntax:	Mcu_RamStateType Mcu_GetRamState(void)	
Service ID:	0x0A	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	Mcu_RamStateType	MCU_RAMSTATE_INVALID: When the RAM state is not conducive for program execution. MCU_RAMSTATE_VALID: When the RAM state is conducive for program execution
Description:	This provides the actual status of the microcontroller Ram. The ram state is reported for the Core from which the API is invoked	
Caveats:	This function requires an execution of Mcu_Init () before it can be used.	
Configuration:	This service is only available if enabled by the pre-processor switch MCU_GET_RAM_STATE_API.	
DET:	MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before this API is used.	
DEM:	No DEM for this API.	
Safety Error reporting	None	
Implementation Comments:	1. The MCU module’s environment shall call this function only if the MCU module has been already initialized using the function Mcu_Init. 2. This function returns the RAM state of available CPUs through the SCU_RSTCON2.CSS register bits.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.	

6.2.11 Mcu_ClearColdResetStatus

Table 179 Specification of API Mcu_ClearColdResetStatus

Service name:	Mcu_ClearColdResetStatus
Syntax:	void Mcu_ClearColdResetStatus (void)
Service ID:	NA
Sync/ Async:	Synchronous

Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This API clears the content of SCU_RSTSTAT register	
Caveats:	This function requires an execution of Mcu_Init () before it can be used.	
Configuration:	This service is only available if enabled by the pre-processor switch MCU_CLR_COLD_RESET_STAT_API.	
DET:	MCU_E_UNINIT (15) is reported when Mcu_Init () is not executed before this API is used.	
DEM:	No DEM for this API.	
Safety Error reporting	None	
Implementation Comments:	1. The MCU module's environment shall call this function only if the MCU module has been already initialized using the function Mcu_Init.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeRuntimeApiEnable is 'False'.	

6.2.12 Mcu_GetVersionInfo

Table 180 Specification of API Mcu_GetVersionInfo

Service name:	Mcu_GetVersionInfo	
Syntax:	#define Mcu_GetVersionInfo (VersionInfoPtr)	
Service ID:	0x09	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Std_VersionInfoType	VersionInfoPtr
Parameters (out):	None	
Return value:	None	
Description:	This API provides the following details : 1. MCU_MODULE_ID 2. MCU_VENDOR_ID 3. MCU_SW_MAJOR_VERSION 4. MCU_SW_MINOR_VERSION 5. MCU_SW_PATCH_VERSION	
Caveats:	None	
Configuration:	None	
DET:	MCU_E_PARAM_POINTER (16) is reported when VersionInfoPtr is a NULL pointer	
DEM:	No DEM for this API	
Safety Error	None	

reporting	
Implementation Comments:	This API is implemented as a macro as per Autosar.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.

6.2.13 Mcu_ClockFailureNotification

Table 181 Specification of API Mcu_ClockFailureNotification

Service name:	Mcu_ClockFailureNotification	
Syntax:	void Mcu_ClockFailureNotification (Void)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This API reports the DEM on clock source failure notification. SMU has to be configured properly so that, this API is called by the SMU on any clock source failure.	
Caveats:	None	
Configuration:	This service is enabled only when the preprocessor switch MCU_E_CLOCK_FAILURE_DEM_REPORT is defined to MCU_ENABLE_DEM_REPORT.	
DET:	None	
DEM:	MCU_E_CLOCK_FAILURE	
Safety Error reporting	None	
Implementation Comments:	None	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.	

6.2.14 Mcu_InitCheck

Table 182 Specification of API Mcu_InitCheck

Service name:	Mcu_InitCheck	
Syntax:	Std_ReturnType Mcu_InitCheck (const Mcu_ConfigType *ConfigPtr)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	ConfigPtr	Pointer to the configuration structure
Parameters (out):	None	
Return value:	E_OK – Initialization comparison successful E_NOT_OK – Initialization not properly done by MCU driver	
Description:	This routine verifies the initialization done by the MCU driver. This should be called after Mcu_Init(), Mcu_InitClock(), Mcu_GetPllStatus() and Mcu_DistributePllClock(). It verifies GTM if GTM_USED is ON, DMA if MCU_DMA_USED is ON and FCE if MCU_CRC_HW_USED is ON.	
Caveats:	None.	
Configuration:	This service is enabled only when the preprocessor switch MCU_SAFETY_ENABLE and MCU_INITCHECK_API is defined to STD_ON.	
DET:	None	
DEM:	None	
Safety Error reporting	None	
Implementation Comments	To reduce code complexity, each comparison will be done by a XOR of value against the inverted value of the intended value or vice versa. The comparison result should always be 0xFFFFFFFF (all 1s). This will also improve runtime since multiple checks are avoided if no failures exist. There is a very less probability of failure happening and can be reported well within the diagnostic test interval; hence this decision.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeInitApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeInitApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeInitApiEnable is ‘False’.	

6.2.15 Mcu_GetMode

Table 183 Specification of API Mcu_GetMode

Service name:	Mcu_GetMode
Syntax:	Mcu_ModeStateType Mcu_GetMode

	(uint8 CoreId)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	CoreId	Core for which the mode has to be determined
Parameters (out):	None	
Return value:	The power state of the core else MCU_UNDEFINED_MODE if CoreId is not within the range	
Description:	This service returns the power mode for the respective core. This API is useful for determining if the desired CPU has gone to IDLE state. This API can be called from other core and check if the desired core has gone to IDLE state.	
Caveats:	McuIdleModeCpuCore should be configured as “INDIVIDUAL_CORES”	
Configuration:	This service is enabled only when the preprocessor switch MCU_SAFETY_ENABLE and MCU_GETMODE_API is defined to STD_ON.	
DET:	None	
DEM:	None	
Safety Error reporting	None	
Implementation Comments	None	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.	

6.2.16 Mcu_RampToBackUpClockFreq

Table 184 Specification of API Mcu_RampToBackUpClockFreq

Service name:	Mcu_RampToBackUpClockFreq	
Syntax:	Std_ReturnType Mcu_RampToBackUpClockFreq (void)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	E_OK – Ramp down to Back up clock frequency successful E_NOT_OK – Ramp down not possible	

Description:	• This function shall be called from CPU Core 0 only. • This service is responsible to ramp down to back up clock frequency (nearest possible frequency which is less than Back up clock frequency). This function is useful before entering standby mode to prevent harmful current drop because during Standby mode, back up clock frequency is used. • This function returns E_NOT_OK when invoked in Pre-scalar mode.
Caveats:	If the current PLL frequency is already below the Back-up clock frequency, then this API will just return without doing anything.
Configuration:	This service is enabled only when the preprocessor switch MCU_RAMP_TO_BACKUP_FREQ_API is defined to STD_ON.
DET:	None
DEM:	None
Safety Error reporting	None
Implementation Comments	This function ramps close to the backup clock frequency in the same number of configured steps as ramping to PLL frequency was done in Mcu_DistributePllClock().
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.

6.2.17 Mcu_ChkWkpStdby

Table 185 Specification of API Mcu_ChkWkpStdby

Service name:	Mcu_ChkWkpStdby
Syntax:	uint32 Mcu_ChkWkpStdby (void)
Service ID:	NA
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	The status of wakeup bits
Description:	This service is responsible to indicate which wakeup event caused an exit from Standby mode.
Caveats:	None
Configuration:	None
DET:	None
DEM:	None
Safety Error reporting	None

Implementation Comments	None
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.

6.2.18 Mcu_ClrWkpStdby

Table 186 Specification of API Mcu_ClrWkpStdby

Service name:	Mcu_ClrWkpStdby	
Syntax:	void Mcu_ClrWkpStdby (uint32 WkpBits)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	WkpBits	Wakeup status bits to be cleared
Parameters (out):	None	
Return value:	None	
Description:	This service is responsible to clear the wakeup events which caused an exit from Standby mode.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Safety Error reporting	None	
Implementation Comments	If MCU_SAFETY_ENABLE is ON, then this function will not invoke Mcal_ResetSafetyENDINIT_Timed to access SCU_PMSWSTATCLR register. If MCU_SAFETY_ENABLE is ON, then User has to call Mcal_ResetSafetyENDINIT_Timed() before calling Mcu_ ClrWkpStdby(). (Refer MCALLIB user manual for Mcal_ResetSafetyENDINIT_Timed api description **[10]).**	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.	

6.2.19 Mcu_SetStandbyCtrlReg

Table 187 Specification of API Mcu_SetStandbyCtrlReg

Service name:	Mcu_SetStandbyCtrlReg	
Syntax:	void Mcu_SetStandbyCtrlReg (uint32 PMSWCR0_RegVal)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non-reentrant	
Parameters (in):	PMSWCR0_RegVal	PMSWCR0 register value
Parameters (out):	None	
Return value:	None	
Description:	This service is responsible for runtime configuration of the Standby mode. It is useful to configure the wakeup events desired at a particular time.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Safety Error reporting	None	
Implementation Comments	If MCU_SAFETY_ENABLE is ON, then this function shall not invoke Mcal_ResetSafetyENDINIT_Timed to access SCU_PMSWCR0 register. If MCU_SAFETY_ENABLE is ON, then User has to call Mcal_ResetSafetyENDINIT_Timed() before calling Mcu_ SetStandbyCtrlReg (). (Refer MCALLIB user manual for Mcal_ResetSafetyENDINIT_Timed api description **[10]).** Note: Tristate enable (TRISTREQ) and Standby RAM supply (STBYRAMSEL) in SCU_PMSWCR0 will be overridden by Mcu_SetMode for Standby.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is ‘True’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is ‘False’ and McuUserModeRuntimeApiEnable is ‘False’.	

6.2.20 Mcu_DeInit

Table 188 Specification of API Mcu_DeInit

Service name:	Mcu_DeInit
Syntax:	void Mcu_DeInit (void)
Service ID:	NA
Sync/ Async:	Synchronous

Origin	Vendor specific
Re-entrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This service is responsible for de-initialization of Mcu driver. This api does not de-initialize the PLL registers. It only de-initializes registers configured by Mcu_Init. Mcu_DeInit will call Gtm_DeInit if Gtm driver is configured. Refer Gtm driver user manual for Gtm de-initialization details.
Caveats:	None
Configuration:	None
DET:	None
DEM:	None
Safety Error reporting	None
Implementation Comments	This api will also de-initialize GTM, CRC and DMA if they are configured. It is assumed that all the drivers will be in de-initialized state when Mcu_DeInit is called.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter McuRunningInUser0Mode is 'True' and McuUserModeDeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeDeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter McuRunningInUser0Mode is 'False' and McuUserModeDeInitApiEnable is 'False'.

6.2.21 Mcu_17_CcuconRegUpdate

Syntax:	void Mcu_17_CcuconRegUpdate (uint8 RegIndex, uint8 Value, uint32 DelayCtr)	
Service ID:	0x52	
Sync/Async:	Synchronous	
Origin	Vendor specific	
Reentrancy:	Non *Reentrant*	
Parameters (in):	RegIndex	CCUCONx(x=6 to 8) Register selection, Allowed values (6,7, and 8) RegIndex value is 6 then CCUCON6 register selected RegIndex value is 7 then CCUCON7 register selected RegIndex value is 8 then CCUCON8 register selected
	Value	CPUx (x = 0 to 2) Frequency Divider Value Range: 0 to 63.
	DelayCtr	Delay counter value (Range: 0 to 0xFFFFFFFF)
Parameters (out):	None	
Return value:	None	

Description:	This service sets the CPUx (x = 0 to 2) Divider value to corresponding CCUCONx (x = 6 to 8) register based on the Register Index value passed as an argument. User configured delay(in Ticks) is introduced followed by Register Write operation to reach fCPUx (x = 0 to 2) to desired frequency. Performing read operation on selected CCUCONx (x = 6 to 8) register to ensure that data is written correctly.
Caveats:	None
Configuration:	None
DET:	1. MCU_E_UNINIT: Report when Mcu_Init is not executed. 2. MCU_E_PARAM_CONFIG: Report when CPUx divider value is greater than 63U.
DEM:	None
Safety Error	MCU_E_PARAM_CONFIG: 1. Report when invalid RegIndex is passed (<6 and >8) 2. Divider Value which is written is not same as read value from CUCONx (x= 6 to 8) Register.
Implementation Comments:	This API update the CPUx (x = 0 to 2) Divider value into corresponding CCUCONx (x = 6 to 8) register based on RegIndex value.
I/O Mode:	This API can be used after Mcu_Init() and before calling Mcu_InitClock(). This API Access should be configured only INIT Time.

6.3 Interrupt Handling

No interrupts are handled by Mcu Driver.

7 Error Classification

The list of the error reported by MCU are defined as mentioned in **Table 189**

Table 189 Error Classification

Error Code	Error Description	Development Error Reporting (DET)	Safety (Production) Error Reporting	Production Error Reporting (DEM)	Value (hex)
MCU_E_PARAM_CONFIG	API Service Mcu_Init called with wrong configuration parameter value	Yes	Yes	No	0x0A
MCU_E_PARAM_CLOCK	API service Mcu_InitClock called with wrong Clock Setting parameter value	Yes	Yes	No	0x0B
MCU_E_PARAM_MODE	API service Mcu_SetMode called with wrong Mode parameter value	Yes	Yes	No	0x0C
MCU_E_PARAM_RAMSECTION	API service Mcu_InitRamSection called with wrong RAM section parameter value	Yes	Yes	No	0x0D
MCU_E_PLL_NOT_LOCKED	Mcu_DistributePllClock() called when the status of the PLL is detected as not locked	Yes	Yes	No	0x0E
MCU_E_UNINIT	APIs called without Mcu_Init being called	Yes	No	No	0x0F

MCU_E_PARAM_POINTER	API service Mcu_GetVersionInfo is called with a NULL_PTR as function parameter.	Yes	No	No	0x10
MCU_E_CLC_ERROR	Enabling of CLC for GTM / DMA / CRC fails in Mcu_Init	No	Yes	No	0x20
MCU_E_OSC_FAILURE	Reported in case of Oscillator failure	No	No	Yes	Assigned by DEM
MCU_E_CLOCK_FAILURE	Reported for clock failure. (invoked by SMU)	No	No	Yes	Assigned by DEM
MCU_E_ERAY_TIMEOUT	Reported in case of ERAY PLL VCO lock timeout	No	No	Yes	Assigned by DEM

There are no additional errors implemented corresponding to specific hardware properties.

8 Example Usage

This chapter describes how the MCU driver can be configured and how to use different API provided by it.

8.1 Configuration of the Driver

To configure MCU driver the following guidelines should be followed properly

1. No Interrupt Priority Initialization required in MCU driver as there are no interrupts
2. MCU driver doesn't depend on any other Modules.
3. Select the appropriate values for the Mcu configurations :
   - MCU General configuration
   - MCU Module Configuration
   - MCU RAM Sector Setting Configuration
   - MCU Clock Setting Configuration
   - MCU Module ModeSetting Configuration
   - MCU Safety Configuration

8.1.1 Sample Configuration of Driver

Let's consider a sample configuration for MCU driver which goes as follows.

- Compiler switch settings
  - MCU_DEV_ERROR_DETECT = TRUE.
  - MCU_VERSION_INFO_API = TRUE
  - MCU_PERFORM_RESET_API = TRUE
  - GTM_USED = TRUE
  - MCU_GET_RAM_STATE_API = TRUE
  - MCU_CLR_COLD_RESET_STAT_API = TRUE
  - MCU_SAFETY_ENABLE = FALSE
  - MCU_RUNNING_IN_USER_0_MODE_ENABLE = FALSE
  - MCU_USER_MODE_INIT_API_ENABLE = FALSE
  - MCU_USER_MODE_DEINIT_API_ENABLE = FALSE
  - MCU_USER_MODE_RUNTIME_API_ENABLE = FALSE

- External oscillator frequency (MCU_MAIN_OSC_FREQ) has to be configured which is equal to crystal frequency in MHz. e.g. MCU_MAIN_OSC_FREQ = 20MHz.

- Clock configuration to generate a 160 MHz PLL clock (Normal Mode)

- RAM section configuration
  - RAM section base address : 0x70000000
  - RAM section with default value 0x0,
  - RAM size of 8192 bytes starting from 0x70000000 address

- Low power modes are already pre-configured in the configuration file for Idle and Sleep modes (MCU mode setting configuration containers are preconfigured as MCU _IDLE and MCU _SLEEP. One of these can be used as ID/parameter to address either IDLE or SLEEP MODE configuration of MCU).

Code Listing 1 Example for Mcu configuration header file.

```
/* MCU module instance ID*/
#define MCU_MODULE_INSTANCE             ((uint8)0)

/*
              Container : McuGeneralConfiguration
              Multiplicity: 1 -1
*/
                    /* Total no. of config sets */

#define MCU_CONFIG_COUNT   (1U)

/*
   Configuration: MCU_DEV_ERROR_DETECT  MCU118, MCU013, MCU100
   Preprocessor switch for enabling the development error detection and
   reporting.
   ON  : DET is Enabled
   OFF : DET is Disabled
*/
#define MCU_DEV_ERROR_DETECT  (STD_ON)

/*
   Configuration: MCU_VERSION_INFO_API MCU118_Conf , MCU168_Conf
   Preprocessor switch to enable / disable the API to read out the modules
   version information.
   ON  : version info API is enabled
   OFF : version info API is disabled
*/

#define MCU_VERSION_INFO_API (STD_ON)

/*
    Configuration : MCU_PERFORM_RESET_API  MCU118, MCU055, MCU167
    Preprocessor switch to enable / disable the use of the
    function Mcu_PerformReset()
    ENABLED  : Mcu_PerformReset function is available
    DISABLED : Mcu_PerformReset function is not available
*/
#define  MCU_PERFORM_RESET_API   (STD_ON)

/*
    MCU181_Conf:
    Configuration : MCU_GET_RAM_STATE_API (McuGetRamStateApi)
    Pre-processor switch to enable/disable the API Mcu_GetRamState.
    (If the H/W does not support the functionality, this parameter
    can be used to disable the Api).
    Since this feature is not present in the hardware, it is permanently
    stuck at STD_OFF.
```

```
*/
#define MCU_GET_RAM_STATE_API (STD_ON)

/*
    Configuration : MCU_CLR_COLD_RESET_STAT_API (McuClearColdResetApi)
    Pre-processor switch to enable/disable the API Mcu_ClearColdResetStatus.
*/
#define  MCU_CLR_COLD_RESET_STAT_API (STD_ON)

/*
    Configuration : MCU_DEINIT_API (McuDeInitApi)
    Pre-processor switch to enable/disable the API Mcu_DeInit.
*/
#define  MCU_DEINIT_API (STD_OFF)


/*
Configuration: MCU_RUNNING_IN_USER_0_MODE_ENABLE
- if STD_ON, enable User0 mode
- if STD_OFF, enable User1 mode
*/
#define MCU_RUNNING_IN_USER_0_MODE_ENABLE (STD_OFF)

/*
Configuration: MCU_USER_MODE_INIT_API_ENABLE
- if STD_ON, Enable Protected Mode(user mode) in Init API
- if STD_OFF, Enable Supervisor mode in Init API
*/
#define  MCU_USER_MODE_INIT_API_ENABLE (STD_OFF)

/*Configuration: MCU_USER_MODE_DEINIT_API_ENABLE
- if STD_ON, Enable Protected mode in DeInit API
- if STD_OFF, Disable Protected mode in DeInit API
*/
#define  MCU_USER_MODE_DEINIT_API_ENABLE (STD_OFF)

/*Configuration: MCU_MODE_RUNTIME_API_ENABLE
- if STD_ON, Enable Protected mode in APIs other than Init/Deinit APIs
- if STD_OFF, Disable Protected mode in APIs other than Init/Deinit APIs
*/
#define  MCU_USER_MODE_RUNTIME_API_ENABLE (STD_OFF)

/*
    Configuration : MCU_SAFETY_ENABLE
    Pre-processor switch to enable/disable the MCU driver safety features.
*/
#define MCU_SAFETY_ENABLE (STD_OFF)


/*
    Configuration : MCU_FMPLL_ENABLE (McuFmPllEnable)
    Pre-processor switch to enable/disable the PLL frequency modulation.
*/
#define MCU_FMPLL_ENABLE (STD_OFF)

/* Reset value of PMSWCR0 */
#define MCU_PMSWCR0_DEINIT_VALUE             ((uint32)0x000002D0U)

/*
```

```
    Configuration : MCU_ENABLE_CLOCK_MONITORING (McuClockMonitoringEnable)
    Pre-processor switch to enable/disable clock monitoring safety measure.
*/
#define MCU_ENABLE_CLOCK_MONITORING (STD_OFF)
/*
    Configuration : MCU_DISABLE_IRADIS (McuIdleReqAckSeqDisable)
    Pre-processor switch to enable/disable IRADIS bit in PMSWCR1.
*/
#define MCU_DISABLE_IRADIS (STD_ON)

/*
    Configuration : MCU_RAMP_TO_BACKUP_FREQ_API (McuRampToBackupFreqApi)
    Pre-processor switch to enable/disable the API Mcu_RampToBackUpClockFreq.
*/
#define MCU_RAMP_TO_BACKUP_FREQ_API (STD_ON)

/*
    Configuration : MCU_SET_STANDBY_CONTROL_API (McuSetStandbyWakeupControlApi)
    Pre-processor switch to enable/disable the API Mcu_SetStandbyCtrlReg.
*/
#define MCU_SET_STANDBY_CONTROL_API (STD_OFF)

/*
    Configuration : MCU_INIT_CLOCK
    Preprocessor switch to enable / disable the use of the
    function Mcu_InitClock()
    ENABLED  : Mcu_InitClock function is available
    DISABLED : Mcu_InitClock function is not available
*/
#define  MCU_INIT_CLOCK (STD_ON)


/* Configuration : McuPBFixedAddress
   This parameter enables the user to switch ON/OFF the
   PostBuild Fixed Address Feature
*/
#define MCU_PB_FIXEDADDR                (STD_OFF)

/*
    Configuration: MCU_MAIN_OSC_FREQ  (MHz)
    This parameter defines external Oscillator frequency in MHz
    This parameter should be configured to the external Oscillator frequency
    value
*/
#define MCU_MAIN_OSC_FREQ  (20U)


#define MCU_DELAY_COUNT_DIVIDER   (2U)

/*
    Configuration: MCU_DISABLE_ERAY_PLL
    This parameter defines if ERAY PLL should be enabled/disabled
    ERAY PLL shall be disabled if Oscillator frequency < 16 MHz
*/

#define MCU_DISABLE_ERAY_PLL (STD_OFF)

/*
    Configuration: MCU_BACKUP_FREQ  (MHz)
```

```
    This parameter defines back up frequency in MHz
*/
#define MCU_BACKUP_FREQ    (100U)

/* OSC Frequency Value
    This bit field defines the divider value that generates
    the reference clock that is supervised by the
    oscillator watchdog. fOSC is divided by OSCVAL + 1
    in order to generate fOSCREF.
    Equation is OSCVAL = (Fosc/2.5 - 1). Xpath eq. becomes (10*Fosc/25 - 1)*/
#define MCU_OSCVAL_REG_VALUE  (0x7U)

/* Master Core for triggering System Modes for controller
 * NA for EP platforms (to avoid compilation error on common source code)
*/
#define MCU_SYSTEM_MODE_CORE            ((uint8)0U)

/* Master Core for triggering Idle Mode for controller
* NA for EP platforms (to avoid compilation error on common source code)
*/
#define MCU_IDLE_MODE_CORE              ((uint8)0U)

/*
              Container: GtmConfiguration
              Multilplicity: 0 -1
*/
    /* Check if GTM is configured */

#define MCU_GTM_USED (STD_ON)
```

Code Listing 2 Example Configuration for MCU driver.

```
/*******************************************************************************
**                        Includes                                           **
*******************************************************************************/

/* Include module header file */
#include "Mcu.h"
/*******************************************************************************
**                      Private Macro Definitions                            **
*******************************************************************************/

/*
   Configuration Options:
   For the clock setting configuration
   PLL status
*/

/* Normal Mode */
#define MCU_NORMAL_MODE        (0x01U)

/*******************************************************************************
**                      Global Constant Definitions                          **
*******************************************************************************/

/* Memory Mapping the configuration constant */
```

```
#define MCU_START_SEC_POSTBUILDCFG
#include "MemMap.h"

/*
   Configuration : Mcu_ConfigType
   For Mcu_ConfigType: MCU specific configuration ID
   This id corresponds to the container name of the McuConfiguration.
   Use this as the parameter for API: Mcu_Init
    - MCU035 : Configuration of MCU
*/
/*
                      Container: McuClockSettingConfig
                      Multilplicity : 1 - *
*/

static const Mcu_ClockCfgType Mcu_kClockConfiguration0[] =
{
    /*McuClockSettingConfig_0*/
  {
    {
      /*K2 divider steps*/
      3U, /* 120.0 MHz */
      3U, /* 120.0 MHz  */
      3U, /* 120.0 MHz  */
      2U,  /* 160.0 MHz */
      0U,
      0U,
      0U,
      0U,
    },
    {
      300U,  /* ramp up delay for K2 div step 1*/
      300U, /* ramp up delay for K2 div step 2*/
      300U, /* ramp up delay for K2 div step 3*/
      400U, /* ramp up delay for K2 div step 4*/
      0U, /* ramp up delay for K2 div step 5*/
      0U, /* ramp up delay for K2 div step 6*/
      0U, /* ramp up delay for K2 div step 7*/
      0U, /* ramp up delay for K2 div step 8*/
    },

    {
      1U,                               /* K1DIV */
      5U,                               /* K3DIV */
      47U,                              /* NDIV */
      1U,                               /* PDIV */
      2U,                               /* K2DivSteps */
      MCU_NORMAL_MODE,                  /* NORMAL/PRESCALER MODE */
      0U                                /* reserved */
    },
    {
      19U,                              /* PLL ERAY N divider */
      4U,                               /* PLL ERAY K2 divider */
      4U,                               /* PLL ERAY K3 divider */
      0U,                               /* PLL ERAY P Divider */
      0U                                /* reserved */
    },
    /*
      SRI Clock     = 160.0MHz
```

```
       SPB Clock     = 80.0MHz
     */
     (uint32)0x02220122U, /* CCUCON0 Register Configuration*/
     (uint32)0x08212212U, /* CCUCON1 Register Configuration*/
     (uint32)0x00000002U, /* CCUCON2 Register Configuration*/
     (uint32)0x00000001U,/* CCUCON5 Register Configuration*/
     (uint32)0x00000020U, /* CCUCON6 Register Configuration*/
     (uint32)0x00000020U, /* CCUCON7 Register Configuration*/
     (uint32)0x00000020U,  /* CCUCON8 Register Configuration*/
     10U, /* Ramp to PLL delay configured in us */

   }/*McuClockSettingConfig_0*/

};


/* Main MCU Configuration Structure */
const Mcu_ConfigType Mcu_ConfigRoot[1] =
{
   {
     /*McuModuleConfiguration_0*/

     /*Mcu Clock Configuration*/
     Mcu_kClockConfiguration0,
     #if (MCU_RAM_SECTORS != 0U)
     /*Mcu Ram Configuration is NULL_PTR as there is nothing configured here */
     NULL_PTR,
     #endif /* #if (MCU_RAM_SECTORS != 0U) */
     /* Reset request configuration  */
     /*
      * Upper 16-bits contain ARSTDIS register configuration value,
      * lower 16-bits contain RSTCON register configuration value
      */
     0x0U,
     /* Total number of Clock settings */
     ((Mcu_ClockType)1U),
     /* Total number of RAM Sectors */
     ((Mcu_RamSectionType)0U),
     /* The maximum mode that is supported by this module instance */
     (MCU_MODE_MAX0),
     /* Ptr to Standby Mode in config structure */
     NULL_PTR,
   }
};

#define MCU_STOP_SEC_POSTBUILDCFG
#include "MemMap.h"
```

8.2 Initialization of Mcu driver

Initialization of MCU driver is done by calling API Mcu_Init with the configuration pointer.

An example below shows call to Mcu_Init API.

Code Listing 3 Usage of Mcu_Init

```
001:    Mcu_Init(&Mcu_ConfigRoot[0]);
```

Note: User must take care that the Mcu_Init API is called before using any of the following APIs: Mcu_InitClock, Mcu_InitRamSection, Mcu_PerformReset, Mcu_SetMode and Mcu_GetVersionInfo. Development error MCU_E_UNINIT is reported if user calls these APIs without calling Mcu_Init

8.3 De-Initialization of Mcu driver

An example below shows call to Mcu_DeInit API.

Code Listing 4 Use Case for Mcu_DeInit

```
001:   /*MCU initialization*/
002:     Mcu_Init(&Mcu_ConfigRoot[0]);
003:   /*MCU De-initialization*/
004:     Mcu_DeInit();
```

Note: User must take care that the Mcu_Init API is called before calling Mcu_Init.

Development error MCU_E_UNINIT is reported if user calls this API without calling Mcu_Init

8.4 Initialization of MCU Clock

MCU Clock can be initialized in one of the two modes,

- Normal Mode
- Prescaler Mode.

Note: Mcu_Init API has to be executed before initializing the clock.

8.4.1 Clock Initialization in Normal mode

PLL will be used to generate the high speed clock. To generate the clock in this mode following steps have to be performed.

Code Listing 5 Use Case for Clock Initialization in PLL mode

```
001:   /*MCU initialization*/
002:     Mcu_Init(&Mcu_ConfigRoot[0]);
003:     /* System Clock Initialization - 160MHz */
004:     RetVal = Mcu_InitClock(0);/*Clock Id*/
005:     if(RetVal == E_OK)
006:     {
007:       while ((Mcu_GetPllStatus()) == 0) {};
008:       Mcu_DistributePllClock();
009:     }
```

For example, to achieve 160 MHz in Normal Mode the following procedure is followed.

Pre-scalar mode is configured and entered. The NMI trap generation for the VCO Lock should be disabled. The first target frequency is back up frequency i.e. 100 MHz The K2 div value is selected in such a way it matches this initial frequency. User defined P and N values are configured. A positive VCO lock status is awaited.

The subsequent K2 div step values are selected in such a way that the resulting frequency is slightly higher than the previous one. The P and N div values remain the same for each updated of K2 div value. The final K2 div step value is the user defined value. The user can configure from a minimum of 0 to a maximum of 6 steps. Between the update of two K2-Divider values 6 CPU cycles should be waited.

By default, two K2 div step values are always available even though the user configures the step as zero. The K2 step value which gives the initial target frequency and the final K2 div step value (user defined) which gives the final target frequency form the 2 divider default step values. In the example given, the user defined number of K2 div step value is 4. Thus, we totally have 6 K2 div steps.

8.4.2 Clock Initialization in Prescaler Mode

To generate the clock in this mode, select PRESCALER_MODE and Program desired K1 divider value in configuration.

System Clock to be generated in this mode by calling single API Mcu_InitClock with the id containing the PRESCALER_MODE in the clock configuration structure

This function initializes the clock in Prescaler Mode

Sequence to initialize the clock in Prescaler Mode is shown in the following **Code Listing 6**. Output Frequency in this module will be equal to the external oscillator frequency.

Code Listing 6 Use Case for Clock Initialization in Prescaler Mode

```
001:   /*MCU initialization*/
002:     Mcu_Init(&Mcu_ConfigRoot[0]);
003:   /* System Clock Initialization – Prescaler Mode*/
004:     retVal = Mcu_InitClock(1);
005:   /*Clock id corresponding to Prescaler Mode*/
```

Note: Execution of Mcu_GetPllStatus will return the value MCU_PLL_UNLOCKED when Clock Initialization is done in PLL Bypass mode.

Note: Mcu_DistributePllClock should not be executed when Clock Initialization is done in Prescaler mode. If this API is executed system clock frequency will be unpredictable.

8.5 Ram section Initialization (Use Case for Mcu_InitRamSection ())

User can initialize the RAM, section wise using this API.

Example usage is shown in the following **Code Listing 7**

Code Listing 7 Use Case for Mcu_InitRamSection

```
001:   /*MCU initialization*/
002:   Std_ReturnType retVal;
003:     Mcu_Init(&Mcu_ConfigRoot[0]);
004:   /* Ram section Initialization */
005:     retVal = Mcu_ InitRamSection (0);/* RAM Section Id*/
006:   /* Return value should be E_OK*/
```

Note: This function requires an execution of Mcu_Init() before it can be used

8.6 Example Usage MCU Reset Reason

In an ECU there are several sources, which can cause a reset. Depending on the type of reset occurred, the application scenarios might look for different application sequence that may be necessary after re-initialization of the MCU. Therefore the MCU driver provides services to get the reset reason of the last reset occurred. This is achieved by executing two APIs.

8.6.1 Mcu_GetResetReason

The user can read the previous occurred reset type (from the hardware) using the API Mcu_GetResetReason (). Mcu_Init () should be called before calling this API.

Example usage of this API is shown in the following **Code Listing 8**

Code Listing 8 Use Case for Mcu_GetResetReason

```
/*MCU ResetReason*/
  Mcu_ResetType ResetStatus;
/*MCU initialization*/
  Mcu_Init(&Mcu_ConfigRoot[0]);
  ResetStatus = Mcu_GetResetReason();
```

8.6.2 Mcu_GetResetRawValue

In case the user wants to read just the content of Reset status register SCU_RSTSTAT register in 32 bit format, Mcu_GetResetRawValue is executed. Mcu_Init () should be called before calling this API.

Example usage of Mcu_GetResetRawValue is shown in the following **Code Listing 9**

Code Listing 9 Use Case for Mcu_GetResetRawValue

```
/*MCU ResetReason*/
  Mcu_RawResetType  ResetRawVal;
/*MCU initialization*/
  Mcu_Init(&Mcu_ConfigRoot[0]);
  ResetRawVal = Mcu_GetResetRawValue();
```

8.6.3 Mcu_ClearColdResetStatus

In case the user wants to clear content of Reset status register SCU_RSTSTAT register in 32 bit format, Mcu_ClearColdResetStatus is executed. Mcu_Init () should be called before calling this API.

Example usage of Mcu_ClearColdResetStatus is shown in the following **Code Listing 10**

Code Listing 10 Use Case for Mcu_ClearColdResetStatus

```
/*MCU ResetReason*/
  Mcu_RawResetType  ResetRawVal;
/*MCU initialization*/
  Mcu_Init(&Mcu_ConfigRoot[0]);
  Mcu_ClearColdResetStatus();
```

8.7 Software Reset (Use Case for Mcu_PerformReset)

There are two types of reset request triggers

- Trigger sources that do not depend on a clock, such as PORST. These triggers force the device into an asynchronous reset assertion independently of any clock. The activation of an asynchronous reset is asynchronous to the system clock, where its de-assertion is synchronized.
- Trigger sources that need a clock in order to be asserted, such as the input signals ESR0, ESR1, WDOG trigger, SMU or the SW trigger.

Note: A PORST reset should only be triggered for power related issues.

- If software detects conditions which require resetting the device, it can perform a soft reset through writing to a special register, the Reset Request (SCU_SWRSTCON) register.

Soft reset is achieved by executing Mcu_PerformReset.

Example Usage of this API is shown in the following **Code Listing 11**

Code Listing 11 Use Case for Mcu_PerformReset

```
001:    /*MCU initialization*/
002:      Mcu_Init(&Mcu_ConfigRoot[0]);
003:    /*MCU Soft Reset*/
004:      Mcu_PerformReset();
```

Note: This function requires an execution of Mcu_Init () before it can be used.

8.8 Low Power Modes

The AURIX power-management system allows software to configure the various processing units so that they automatically adjust to draw the minimum necessary power for the application. There are three power-management modes: Run Mode, Idle Mode, Sleep and Standby Mode. Idle, Sleep and Standby modes will make AURIX system to run on low power.

These low power Modes can be achieved by executing Mcu_SetMode API.

Example usage of Mcu_SetMode is shown in the following **Code Listing 12**

Code Listing 12 Use Case for Mcu_SetMode

```
001:    /*MCU initialization*/
002:      Mcu_Init(&Mcu_ConfigRoot[0]);
003:
004:    /* sleep mode*/
005:    Mcu_SetMode(MCU_SLEEP);
006:
007:    /* idle mode*/
008:    Mcu_SetMode(MCU_IDLE);
009:
010:    /* standby  mode*/
011:    Mcu_SetMode(MCU_STANDBY);
```

8.9 MCU Driver Version Information

MCU driver information can be accessed by calling Mcu_GetVersionInfo API. This service returns the version information of this module. The version information includes:

- Module Id
- Vendor Id
- Vendor specific version numbers

This function is pre compile time configurable On/Off by the configuration parameter: MCU_VERSION_INFO_API.

This function requires an execution of Mcu_Init () before it can be used.

Example usage of Mcu_GetVersionInfo is shown in the following **Code Listing 13**

Code Listing 13 Use Case for Mcu_GetVersionInfo

```
001:    /*MCU initialization*/
002:      Mcu_Init(&Mcu_ConfigRoot[0]);
003:
004:    #if (MCU_VERSION_INFO_API == ON)
005:      Std_VersionInfoType McuVersionInfo;
```

```
006:      Mcu_GetVersionInfo(&McuVersionInfo);
007:    #endif
```

8.10 Mcu_17_CcuconRegUpdate

User can write CPUx (x = 0 to 2) divider values into corresponding CCUCONx(x= 6 to 8) Registers to set the required core frequency to desired value. This API should be called before Mcu_InitClock ()

Example usage of Mcu_ClearColdResetStatus is shown in the following **Code Listing 14**

Code Listing 14 Use Case for Mcu_17_CcuconRegUpdate

```
001:    /*MCU initialization*/
002:      Mcu_Init(&Mcu_ConfigRoot[0]);
003:      Mcu_17_CcuconRegUpdate(6,20,20000); /* writes 20 decimal value
                                                 into CCUCON6 register,
                                                 which sets Core-0 frequency
                                                 to desired value
                                               */
```

8.11 Sample Sequence Diagrams

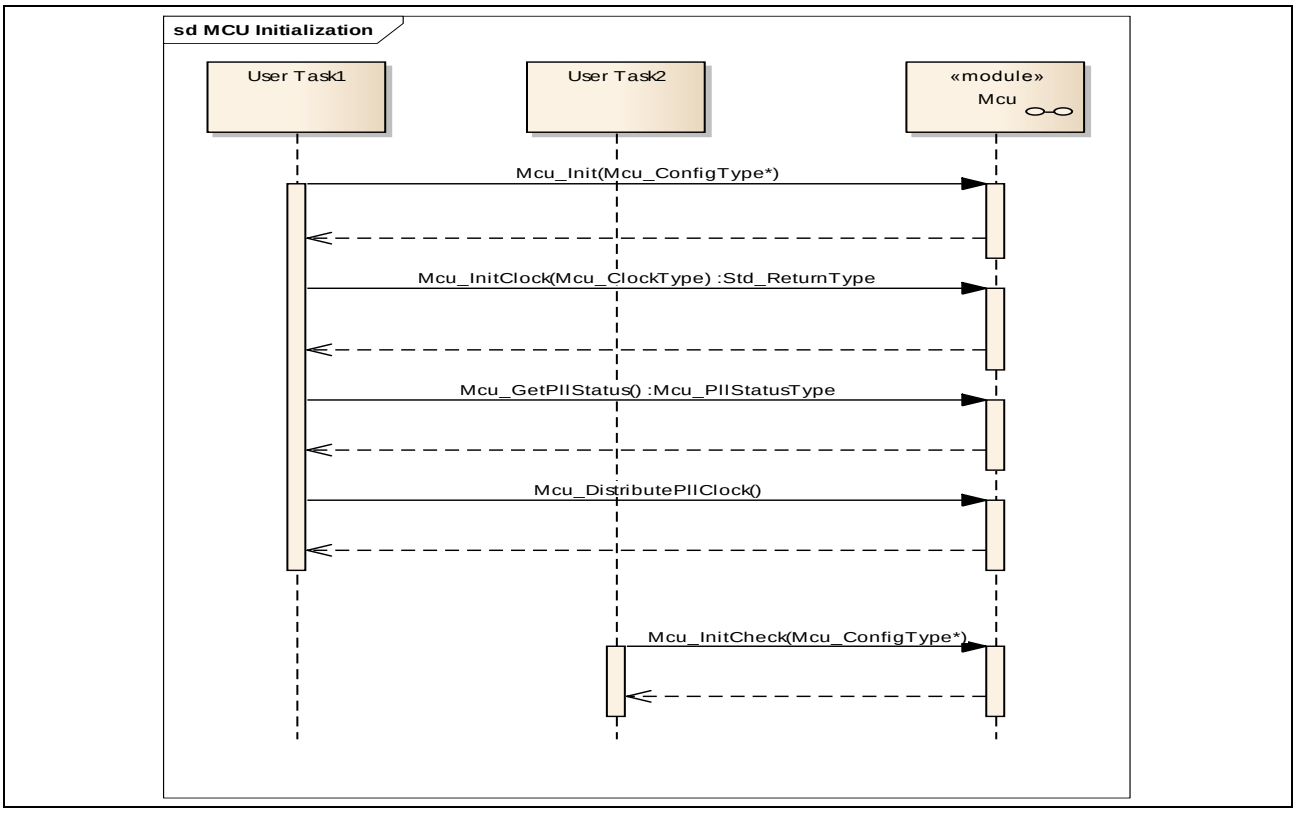

Figure 4 MCU Safe Initialization

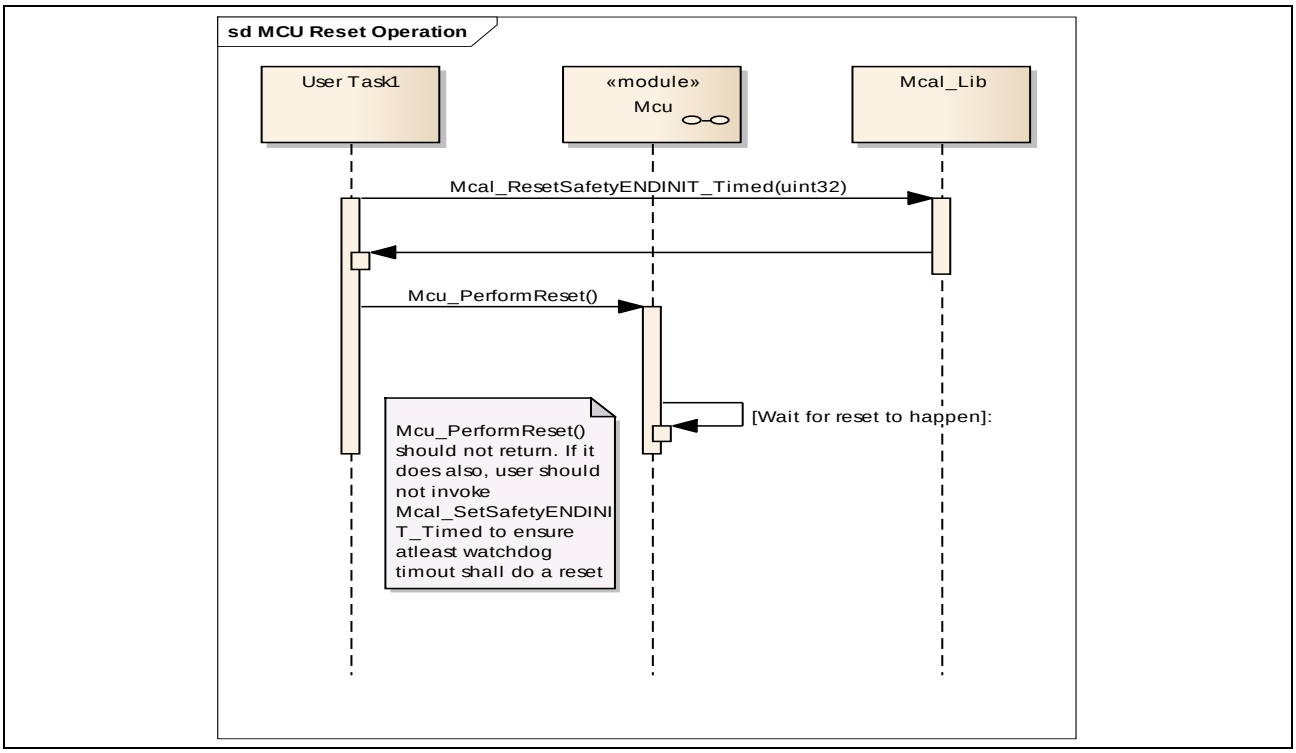

Figure 5 MCU Intentional Software Reset

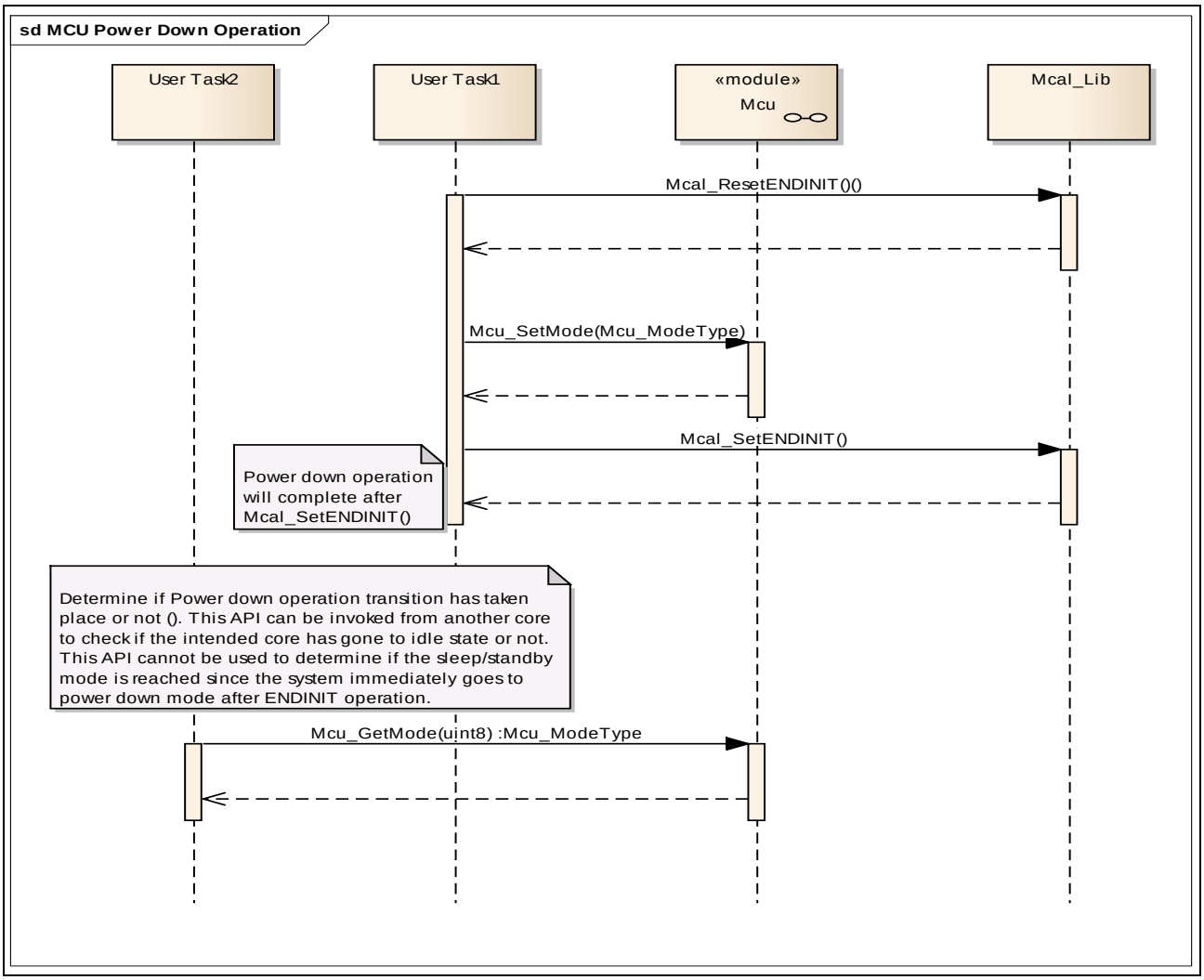

Figure 6 MCU Intentional Power Down to IDLE state

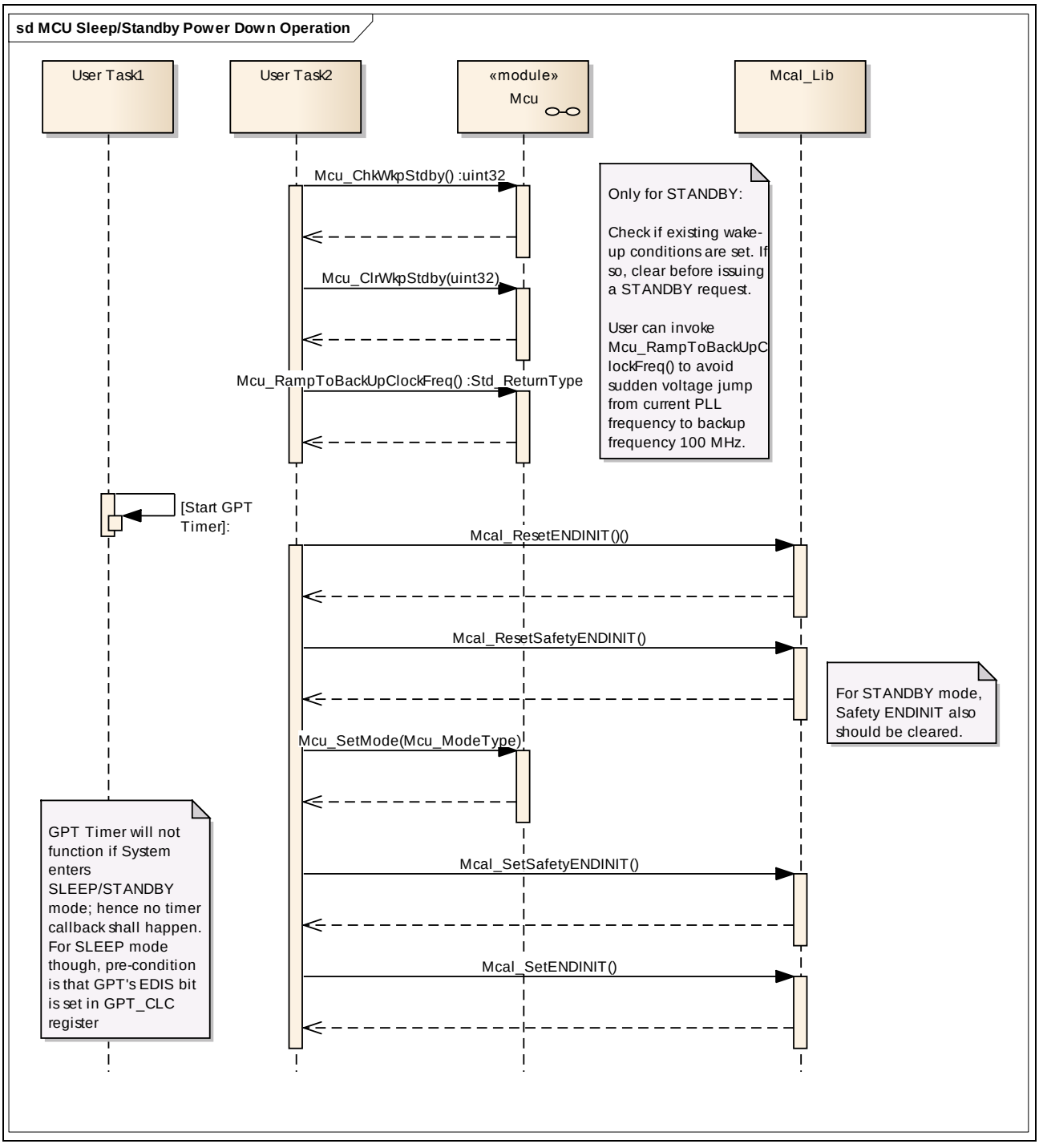

Figure 7 MCU Intentional Power Down to SLEEP/STANDBY state

9 Limitations and Assumptions

9.1 Assumptions and Deviations from the Software Specification

Table 190 Assumptions

Sl. No	Reference	Assumptions

1	General	The global configuration (interrupt arbitration cycle, priority settings) of interrupts will be handled by OS.
2	Multiplicity of container MCU Clock Setting Configuration	Multiplicity of MCU Clock Setting Configuration is taken as 1 - *
3	Callback handling (MCU219)	This requirement is purely for information purpose and applicable to PC variant only.
4	FMPLL Support	This implementation support is not applicable for the TC27X_BC step release
5	McuK2DivRampUpConfDelay and McuK2DivRampDownConfDelay Support	It is assumed that CPU0 will execute Mcu_DistributePllClock, as these delays are calculated based on CPU0 divider.
6	TC22x and TC21x	The maximum frequency supported for TC21x and TC22x is 133.333333MHz.

Table 191 Deviations

Sl. No	Reference	Deviations
1	McuClockSettingId is not considered.	Due to different versions of explanation in Mcu_ClockType and MCU183_Conf, this parameter is omitted and in place of it, symbolic names with range 0...<N-1> are generated and used. Where N is the total number of clock settings that are configured.
2	McuClockReferencePoint container multiplicity fixed at 1:1	McuClockReferencePointFrequency is considered as PLL frequency only; hence McuClockReferencePoint container is fixed at 1:1.
3	Production error handling (MCU015, MCU049, MCU051, MCU053, MCU102, MCU109, MCU111, MCU166, MCU170_Conf, MCU187_Conf, MCU188_Conf, MCU220 and MCU226)	The only production error given by AUTOSAR is clock failure. SMU has to be configured to call the API Mcu_ClockFailureNotification() to report DEM.
4	McuResetSetting configuration parameter is implemented as VariantPB.	McuResetSetting configuration parameter is handled in the multi-configset and hence implemented as VariantPB rather than VariantPC as stated by AUTOSAR (MCU173_Conf).

9.2 Hardware Features Not Supported

1. MCU driver does not support the clock unit to run solely on back-up clock frequency except before entering standby mode.
2. MCU driver does not support Standby mode in unanimous mode (Refer **4.2.8** for more details). This restriction is because Mcu_SetMode () in Standby mode handles the RAM redundancy data which needs to be done only once. But each core calling Mcu_SetMode () in unanimous Standby mode will cause corruption of RAM redundancy data. Refer **6.2.9** for more details. This does not apply for AURIX EP platforms.
3. Modules that make use of ECU resource properties file shall enable the ECU Resource Manager in the module's MANIFEST.MF file.

w w w . i n f i n e o n . c o m

Published by Infineon Technologies AG